

An Experiment in Using AI for Cross-Platform App Development with Flutter

Norah Kuduk

Ed Harcourt

December 20th, 2024

Abstract

A cross-platform physical therapy app was developed to provide accessible, customizable solutions for athletes and individuals managing chronic injuries. The app allows users to schedule routines and view routines using a calendar, as well as customize routines with a robust set of standard exercises and user-input exercises. Flutter and Dart were used to build the app, starting with a minimum viable product that included hard-coded routines, a simple user interface, and data handling capabilities. A PostgreSQL database was used to store information about routines, such as names, exercises, repetitions, and customizations. Docker was implemented for containerization, and the database was deployed within the container, requiring server-side endpoints to access information in the database. AI tools like ChatGPT and GitHub CoPilot assisted in coding, debugging, and problem-solving, and their effectiveness was documented to highlight their use in enhancing work.

USING AI FOR CROSS-PLATFORM APP DEVELOPMENT

Table of Contents

Abstract.....	1
1. Introduction	3
1.1 <i>Overview of App Development and Its Evolution</i>	3
1.2 <i>Relevance of AI in Coding Workspaces.....</i>	4
2. App Development with Flutter.....	5
2.1 <i>Flutter.....</i>	5
2.2 <i>Dart and Asynchronous Programming</i>	6
2.3 <i>High-Level Overview of the App.....</i>	7
3. The Physical Therapy App Structure.....	8
3.1 <i>The Minimum Viable Product (MVP)</i>	8
3.2 <i>The Relational Database.....</i>	8
3.3 <i>Server-Side Access</i>	9
3.4 <i>The Final App</i>	10
3.4.1 Home Screen	11
3.4.2 Hamburger Menu (and Settings Screen).....	12
3.4.3 Routine Detail Screen	13
3.4.4 Calendar Screen.....	14
3.4.5 Exercise List Screen	15
4. AI Usage	15
4.1 <i>Overview of AI Tools.....</i>	15
4.2 <i>AI in Education</i>	16
4.3 <i>AI for Error Checking</i>	17
5. Conclusion.....	17
5.1 <i>Summary of Key Findings.....</i>	17
5.2 <i>Future Work.....</i>	18
6. References	19
7. Appendices	20
7.1 <i>Backend Server Requests</i>	20
7.2 <i>Database Helper</i>	28
7.3 <i>DDL.....</i>	29
7.4 <i>Minimum Viable Product (MVP).....</i>	30
7.5 <i>Main</i>	35
7.6 <i>Home Screen.....</i>	36
7.7 <i>Routine Info Screen</i>	42
7.8 <i>Exercise List Screen.....</i>	53
7.9 <i>Settings Screen</i>	57
7.10 <i>Calendar Screen.....</i>	58
7.11 <i>Schedule Dialog.....</i>	62

1. Introduction

This paper will focus on two primary objectives. The first objective will highlight the comprehensive physical therapy app's development process and the decisions made to create a functioning cross-platform app. The second objective will explore using Artificial Intelligence (AI) in a professional workspace for work efficiency and development assistance.

1.1 Overview of App Development and Its Evolution

Building an app requires a well-defined process that developers use to take a concept for an application to reality. Starting with the ideation, a developer (or a company employing a developer) probably has some idea of the problem or task this app will seek to solve. This will also include identifying the core features that the app will be built around. Next, a developer will design and create a prototype of the app, allowing them to visualize what a user might see and be able to accomplish with an app, but without any of the functionality. Then, a developer will code the app to implement the desired functionality, testing features as they're added [1]. This is typically the most arduous piece of the process, as it involves lots of coding and debugging to implement both necessary and desired features. Once an app is in its final state, it can be deployed to various app stores to be available for download. Work doesn't stop there, though; once deployed, an app has to be continuously updated to maintain functionality across software and other updates that might affect use [5].

There has been a significant increase in the popularity of mobile app development in recent years. Traditionally, app development has used platform-specific tools like Swift or Kotlin to develop iOS and Android. In doing this, developers would need two separate codebases if they wanted their app to be available on both the App Store and the Google Play Store (or any other store) [10]. This was enough for most companies and organizations that needed to develop apps

for a long time. Still, there were some glaring challenges that recent evolutions in app development sought to fix. A few of these challenges included increased development costs and time, as well as complications in keeping apps consistent across platforms. Fixing these challenges was the work of new cross-platform frameworks, like Flutter or React Native.

These platforms allow developers to write a single codebase that can be exported to iOS, Android, or web apps. This saves developers time and companies money, making the process exponentially easier. Cross-platform frameworks also reduce app development's complexity, making it more accessible to the average person. This paper will dive deeper into how effectively a cross-platform framework (with the help of Artificial Intelligence) can be used to rapidly develop a functional app.

1.2 Relevance of AI in Coding Workspaces

With the exponential growth of AI in recent years, a typical thought process (and fear) of many is that, eventually, AI will render many technically based jobs obsolete. This is a significantly complex issue, as, on the one hand, there is a strong argument that, yes, AI will render some jobs redundant. On the other hand, the benefits of AI far outweigh the few negatives it brings.

In the computer science and tech industry, AI tools are becoming increasingly popular in professional software development settings. Rapidly evolving programs assist developers with tasks such as autocompletion, error detection, and code refactoring. They do not replace developers' jobs but, in fact, make them more efficient, allowing them to spend less time on inconsequential tasks and more time on programming. Tools like ChatGPT and GitHub Copilot (utilized in this project) are designed to improve developer productivity by automating repetitive tasks, generating code snippets, and offering solutions to common problems. These

USING AI FOR CROSS-PLATFORM APP DEVELOPMENT

advancements in AI have the potential to significantly transform the software development process, making it faster, more accessible, and more efficient.

Another side effect of the growing prevalence of AI in software development is the popularization of coding in industries beyond technology. This is because AI tools being developed are not just for seasoned programmers. While they've most commonly been used in a tech-driven space, non-technical professionals also use them to contribute to software projects.

One feature of AI is that it generates code snippets and explains complex concepts in simpler terms. Individuals with minimal programming experience can now participate in app development, making coding more inclusive. This also saves time communicating between teams in a corporate workspace, as more time can be spent working rather than trying to ensure everyone is on the same page. This project seeks to better comprehend how AI can make work more efficient in the corporate and academic space.

2. App Development with Flutter

This section will discuss Flutter as a cross-platform app development framework and provide some background on the Dart programming language and how it was used in this project. It will also provide a high-level overview of the fitness app developed and its goals without delving into the technicalities.

2.1 Flutter

Flutter is an open-source framework for building natively compiled mobile, web, and desktop applications from a single codebase [6]. It was first released in 2018 and has since been used to develop over one million apps on many topics and functions.

Some of the key features of Flutter that make it incredibly useful for app development is its implementation of a "hot reload," which allows developers to see changes in real-time without

needing to fully restart the app after changing a small feature [4]. It also provides a robust library of pre-built widgets that developers can utilize in their own apps or use as reference to build their own custom widgets.

Flutter is written in a coding language called Dart, which is a language optimized for client apps. A client application is a software program that connects to remote systems, sends data, and receives responses. Essentially, it is the in-between interface for a user and backend functionality.

2.2 Dart and Asynchronous Programming

Diving further into Dart, its abilities, like support for asynchronous programming and compiling to native code, make it ideal for building apps.

Asynchronous programming allows code to complete other operations while waiting for another operation to finish. In real life, this could be considered everyday household chores that require a significant amount of time but not constant work, such as laundry. Using an asynchronous program, you could start the washing machine and go on with the rest of your day until it notifies you that the cycle is complete. Without it, you'd be unable to complete any other tasks until the cycle was done [2]. In the context of this project, common asynchronous operations include fetching data over a network and writing to a database.

Dart utilizes the **Future** class to represent the results of asynchronous operations, which a future instance can represent. A future instance will, by default, be incomplete when an asynchronous function is called; this instance is the placeholder until the operation finishes or throws an error. This allows other program functionality to continue while the operation finishes [3]. Asynchronous programming is beneficial for client app development, as there are often operations like fetching data that can take significant amounts of time.

2.3 High-Level Overview of the App

The app is designed to provide customizable workout routines and track user progress. It is accessible to a range of users, from athletes recovering from injuries needing highly customizable routines during the recovery process to users with chronic conditions needing to track for longer periods.

Key features of the app include a simple interface for creating and managing routines, a calendar to track and create a schedule for routines, and a robust list of exercises to pick from (along with the option for users to add their own custom exercises).

The app aims to address gaps in existing physical therapy tools, particularly in terms of accessibility and personalization. Using my personal experience in the St. Lawrence University training room, we know that athletes of all different medical backgrounds and sports teams use sheets of paper to keep track of their various rehab and physical therapy routines, which need to be changed often due to the progression of the athletes, which can be confusing for trainers and athletes alike. Users can tailor programs to their needs, add notes for exercises based on customizations or equipment needed, and use a digitally enabled physical therapy exercise database.

This app is also necessary due to the increasing demand for accessible, personalized, and flexible physical therapy solutions. Individuals often seek ways to maintain their physical health or recover from injuries at their convenience and pace. The app aims to bridge the gap between traditional in-person physical therapy sessions and the growing need for at-home therapy options, providing step-by-step guidance tailored to each user's unique situation.

3. The Physical Therapy App Structure

3.1 The Minimum Viable Product (MVP)

The first step in developing any app is determining the Minimum Viable Product (MVP). To define it further, an MVP implements the base features needed for an app to be “functional” for users, but not necessarily have all of the bells and whistles a fully developed app would have.

The MVP is purely functional and not comfortable to use. In our case, to have the app's core functionality, our MVP needed to have a way to store our data, input routines into the database, and have a working UI for users to view routine details.

In the MVP, our data was stored internally within the app, using JSON objects to store our list of exercises needed for the routine. We would use this JSON structure later when determining the relational database and what information tables would need to hold.

The MVP was also written in a single file, reducing the need for code organization and references within the project, but it became extremely messy. The best practice when coding is always to minimize the amount of code within a file or function.

Overall, the working MVP was helpful first in getting a feel for how core Flutter functionalities can be applied to an app and second in theorizing how to best implement the more complex features desired.

3.2 The Relational Database

As storing the data locally and hard-coding routines wouldn't be an optimal method of storage in the long term, we needed to come up with another way to store our data so it would be accessible to the app. We elected to use a PostgreSQL database to store information, routines, exercises, and scheduling for the app to access. An SQL database is a relational model, meaning data is organized into tables, or “relations,” linked together by a common attribute.

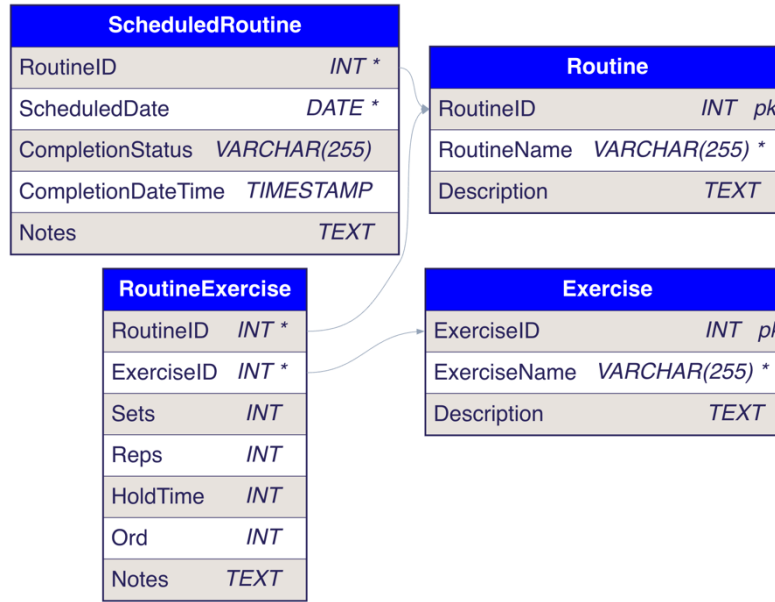


Figure 1. ER Diagram of the database used to store app information

An entity relationship (ER) diagram illustrates the relationships between these tables, ensuring data consistency and integrity. The ER diagram above shows the relational model used to store information for the physical therapy app. The diagram above shows the model as created by the DDL.sql file (referenced in the appendices).

3.3 Server-Side Access

In this project, Docker was used to create a container to ensure that there would be no issues when developing or testing the app on different machines. Docker is an open-source platform used by developers to run isolated development environments, called containers, which include everything an application needs to run. For example, it will have the application code, runtime environment, libraries and dependencies, and configurations. This means the container is fully self-sufficient and can be used across platforms. Docker containers are instances of a Docker image, which is essentially the package of all the little things needed to create the container [23]. In this project, by containerizing the PostgreSQL database, we could ensure that the same version of the database could be used across all instances of the app.

Inside the app, we designed a REST API (Representational State Transfer Application Programming Interface) that allows the Flutter app to communicate with the PostgreSQL database and vice versa. This allowed us to have a clear separation between client-side (app) and server-side (database) logic. A REST API is a way to allow software applications to communicate with each other over the internet, which is standard across many development practices [12].

A REST API focuses on resources and has consistent rules for retrieving, creating, and deleting data. In our case, a resource would be considered a routine, an exercise, an exercise in a routine, or a schedule entry. As the REST API has a resource-based design, it provides a way for users to interact with it using URLs. For example, in this project, the command **curl -X GET http://localhost:8080/routine** would return a JSON object of all of the entries in the routine table in the database. As a general rule, REST APIs use HTTP methods to interact with resources (GET to retrieve, POST to create new, PUT to update, and DELETE to delete) [13].

To access information from the database, we implemented endpoints for each CRUD (Create, Read, Update, Delete) operation the app needs. For example, a create command would be adding a new routine or exercise, a read could be getting a list of all routines created, and an update could be adding an exercise to an existing routine. Each of these endpoints will call the respective HTTP operation to be used in the server-side logic.

3.4 The Final App

This project utilized Flutter's extensive widget library to create a user-friendly interface. Dart code was used to implement a connection to the database and manage the app state. The app used specific Flutter/Dart packages, like **shelf** and **http**, for API integration, which played a critical role in its development [10].

The following sections detail each “screen” of the app, in addition to a description of the operations that a user can perform. Each of these “screens” is coded within its own file, referenced in the appendices section at the end of this paper.

3.4.1 Home Screen

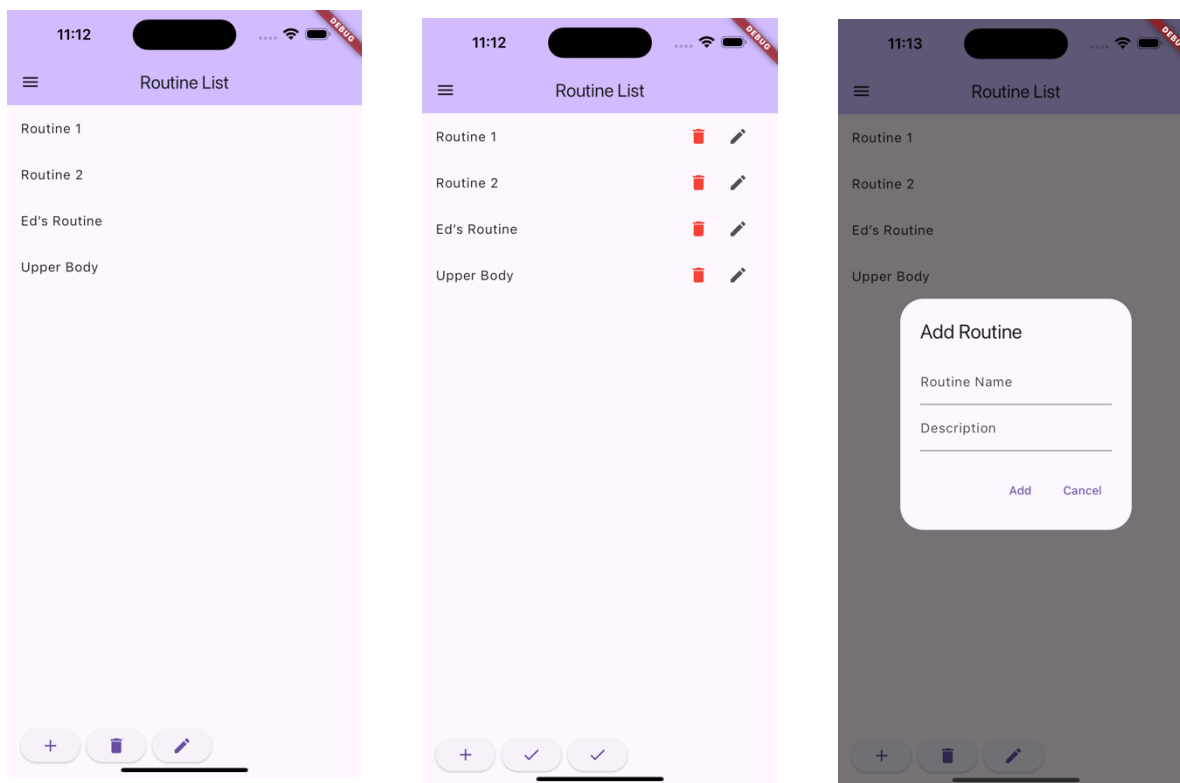


Figure 2. Images of the Routine Page, with options for editing, and adding.

The home screen page is the default screen that is opened when a user enters the app. On this screen, users will be able to view all the routines created and open the hamburger menu (detailed in the next section). Considering the routines, the screen also allows users to edit the routine list by creating a new routine, editing an existing routine, or deleting an existing one.

The “+” icon at the bottom of the screen opens the dialogue to add a new routine. The trash can and pencil icons toggle the buttons to delete or edit routines, then hide them when not in use. This format is the same as that used for the exercise and routine information screens.

3.4.2 Hamburger Menu (and Settings Screen)

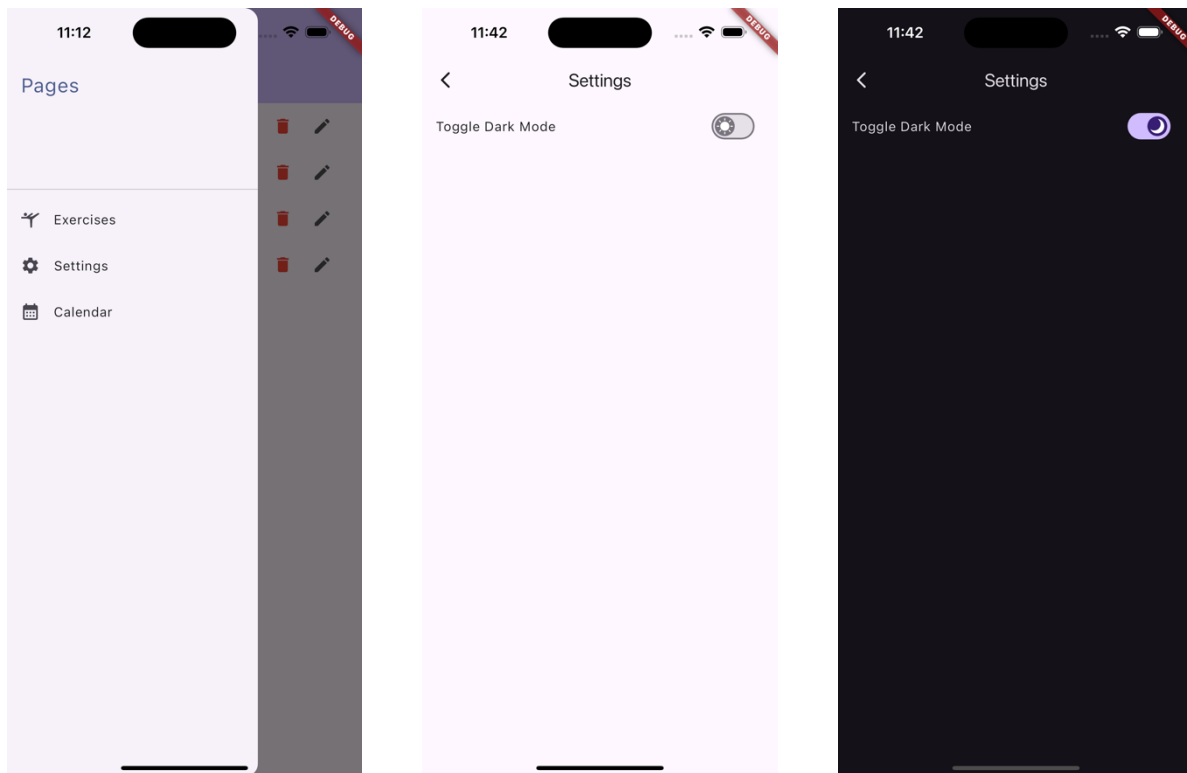


Figure 3. Hamburger Menu, Settings Screen, and showing light/dark mode

The hamburger menu is a screen that opens by clicking on the three lines in the top left corner of the home screen. It opens up a menu to take users to a subpage, either the exercise list, the settings page, or the calendar.

The settings screen will allow users to modify different settings to customize app use. Currently, the only option is switching between light and dark modes. The default will match the system setting of a user's device, but can be overwritten using the toggle. In the future, the settings screen will also have options for font size, overall color, voice type, and more accessibility features.

3.4.3 Routine Detail Screen

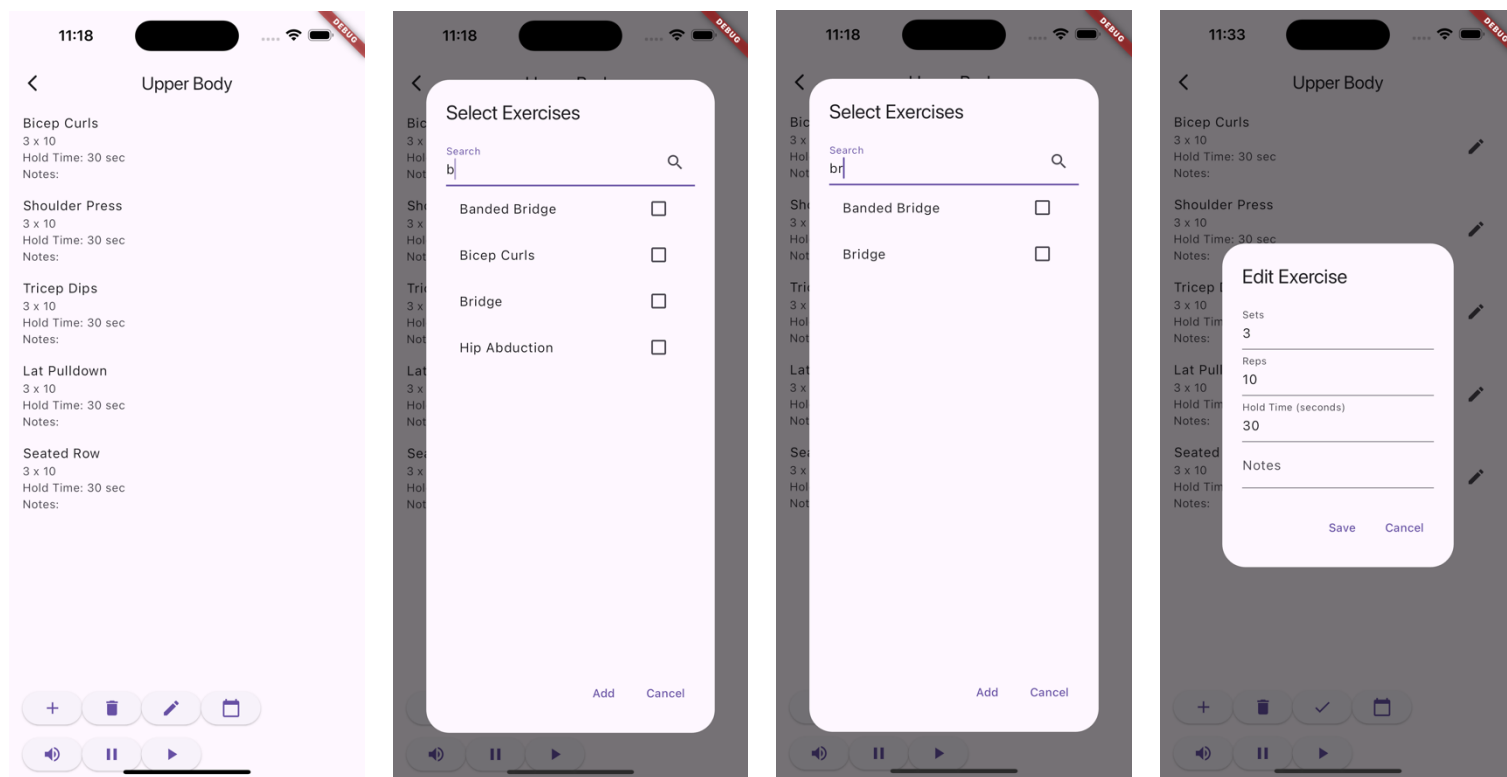


Figure 4. Shows customizing a routine's exercises, dynamic searching, and editing display information

Users are shown the routine detail screen once a routine is clicked on the home screen. This screen shows all of the individual exercises in the routine and allows users to add exercises, modify repetitions/sets, add notes, and more.

It also has a text-to-speech (TTS) feature implemented for accessibility [7]. The TTS allows users to start the speech with the volume button, having a voice walk through each exercise so that a user can complete a whole set of exercises without needing to get up to look at their phone. Users can pause and play with the other two buttons in the row.

When adding exercises to a routine, a dynamic search bar allows users to search the entire database of exercises and update the list as they type. Users can also drag and drop the exercises

to change their order. The calendar icon to the right of the check will open the scheduling dialogue for routines.

3.4.4 Calendar Screen

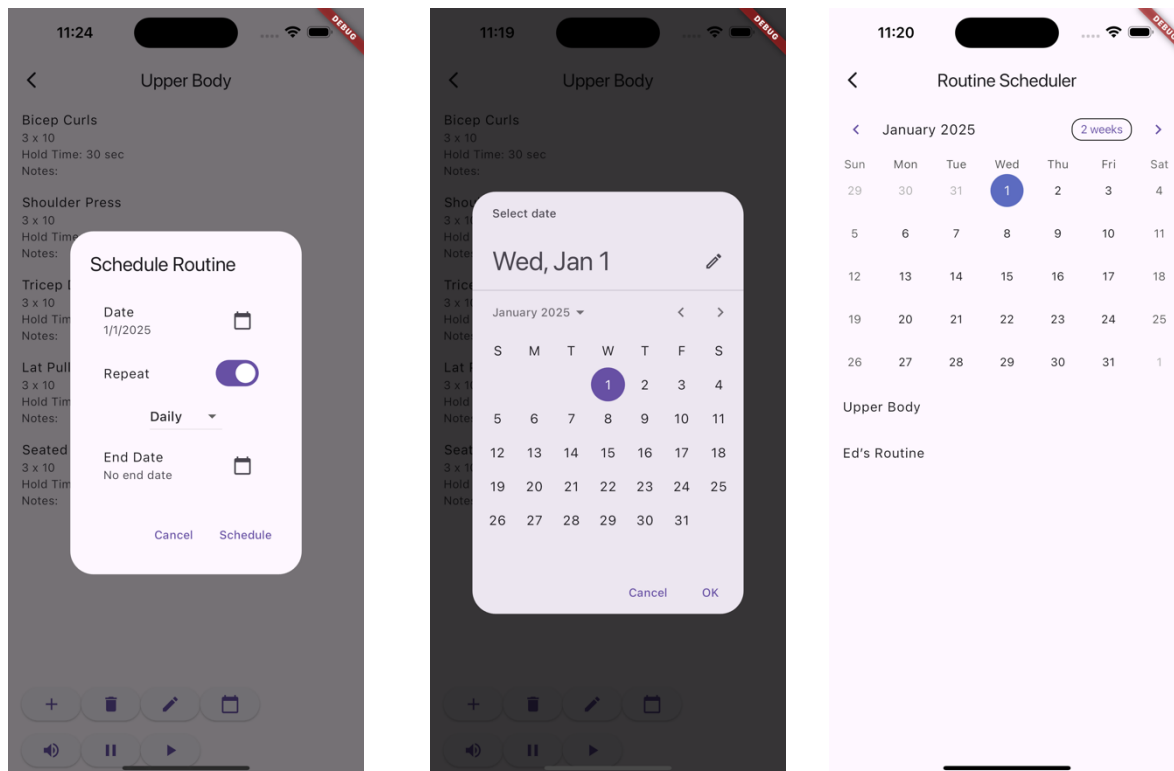


Figure 5. Options to schedule a routine, picking a date, and showing the calendar page with a list of scheduled exercises

The calendar screen page was built using the **table_calendar** public source Flutter package [18]. Users can see different months on the screen and select days to view what routines are scheduled on that day. If no routines are planned, there is also a dialogue to support that.

Routines are added to the calendar from the routine information page. Users can schedule each routine by clicking the calendar icon at the bottom of the screen. Clicking will pop up the scheduled dialogue, where users can pick a date for a single instance of a routine or toggle repeat functionality so that a routine can be scheduled multiple times up to an end date.

3.4.5 Exercise List Screen

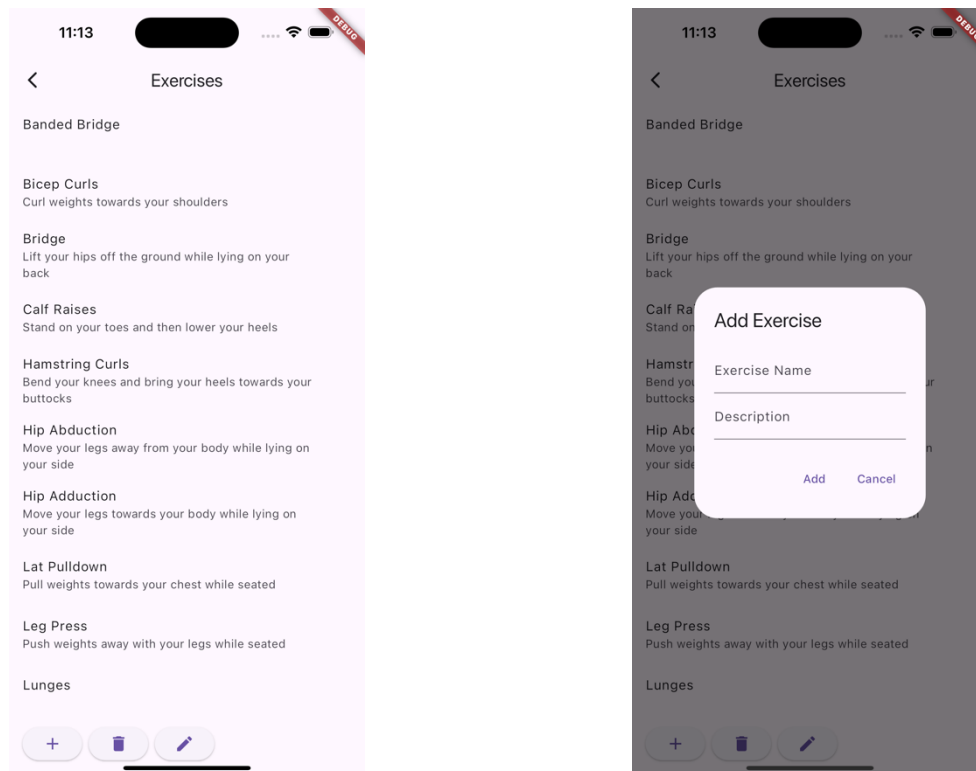


Figure 6. Showing the list of all exercises in the database and adding a custom exercise

The exercise list screen contains an alphabetized list of all of the exercises currently loaded into the database. If an exercise table does not exist (creating a new instance of the app), an SQL file will populate the relation with a default set of common exercises.

In addition to the default set of exercises, users can add their own custom exercises to the list. Once added, any exercise in the list will be displayed in the master list when selecting an exercise to be added to a routine. Similar to the home screen and the routine info screen, the trash can and pencil icons toggle delete and edit buttons for each exercise.

4. AI Usage

4.1 Overview of AI Tools

Much of this project relied on AI technology to improve learning and development. We utilized the capabilities of AI tools like ChatGPT (an external chatbot) [14] and GitHub Copilot

(a VSCode Extension) [8]. ChatGPT generated code snippets, answered technical questions, and provided debugging assistance, whereas GitHub Copilot offered autocompletion and code suggestions within the IDE, speeding up the development process.

Considering the use of AI in this project, one important thing to note is that it cannot do everything for a programmer. While incredibly useful, AI tools would constantly provide incorrect or suboptimal solutions or answer an entirely different question than the one prompted, requiring human intervention to fix or refine the code.

Specific prompts and redirection were needed to successfully utilize AI to enhance code. The only way to use AI in this way successfully requires significant knowledge of the language and the goal of the code being written. This highlights the continued need for human coders and reinforces the notion that AI will not overtake all technically oriented jobs.

4.2 AI in Education

A strong motive for this project is also the growing popularity of AI, and the subsequent questions raised about its role in academia. In universities across the country, many professors have been revising their course syllabi and policies to include AI as a learning tool while also including strict guidelines about acceptable use [20]. It is a fine line to walk, as the benefits of AI are endless, but it is incredibly easy for students to fall into the temptation of overusing it to the point of academic dishonesty. If used too heavily, over-reliance on AI could also hinder students from developing problem-solving skills and a deeper understanding of programming principles.

In Computer Science (and other STEM programs specifically), AI tools can provide personalized support for students, helping them learn new programming concepts and troubleshoot errors more effectively, allowing class time to focus more on learning than singular issues.

4.3 AI for Error Checking

One place where AI support was invaluable during the project was its usefulness in understanding and explaining incorrect code. When creating the calendar view, we ran into many issues with mismatched data types attempting to be accessed, as the **table_calendar** package could only be utilized with **DateTime** types. In order to use the **DateTime** payload, we needed to parse the object for relevant information. ChatGPT [14] was able to help with this by providing it with code from the **server.dart** file where the schedule table was being queried, it was able to provide a snippet of code to parse a **DateTime** object. A typical object is in the form **YYYY-MM-DDTHH:MM:SS**. AI provided the extension **“.toIso8601String().split('T').first”** which would split the string on the T, then take the first half to only keep the relevant date.

While AI tools helped resolve many issues, they could also sometimes produce false positives or overlook more complex problems. In this case, it is important to recognize that while AI is helpful, it cannot replace a human programmer, and also needs to be consistently monitored when being used in a professional space.

5. Conclusion

5.1 Summary of Key Findings

The project demonstrated the feasibility of building a functional cross-platform app using Flutter, Dart, and AI tools, highlighting their potential to accelerate development. Beginning this project with little to no knowledge of app development or the Flutter/Dart framework, the progress made highlights AI tools' effectiveness in enhancing the development process.

Overall, we were able to develop a locally-run, fully functioning fitness app, with a variety of features implemented for ease of use, as well as accessibility for users. The use of AI tools in a

professional workspace enhances the development process, saving valuable time on a short-term project.

5.2 Future Work

As it stands, the app is fully functional, with a robust suite of features that bring it beyond what was initially developed using the MVP. If we were to continue growing the app, there are a few features that we would focus on implementing first.

One would be to add a feature to track the completion status of routines and store summary data. This would allow users to retrieve information about what routines they performed and when quickly. The data could also help users analyze their fitness trends over extended periods.

Going along with data trends, another feature that would be essential if the app were to be deployed would be integration with external health data stored on a user's device (like iOS's Health/Fitness app) to gather a more holistic view of health.

While the app is not in a state to be deployed to either the Apple or Google Play stores, it remains a future goal. In order to do so, the app will need to be further refined and cleaned up, in addition to including more accessibility features (mentioned earlier) and conforming to national and international data privacy guidelines.

6. References

- [1] Application development life cycle: An overview. (n.d.). Retrieved from <https://www.ibm.com/docs/en/zos-basic-skills?topic=zos-application-development-life-cycle-overview>
- [2] “Asynchronous Programming: Futures, Async, Await.” *Dart*, dart.dev/libraries/async/async-await.
- [3] BillWagner. “Asynchronous Programming - C#.” *Asynchronous Programming - C# | Microsoft Learn*, learn.microsoft.com/en-us/dotnet/csharp/asynchronous-programming/.
- [4] Building user interfaces with Flutter. (n.d.). Retrieved from <https://docs.flutter.dev/ui>
- [5] “Expert Web and Mobile App Development Services: Vivaswan Tech.” *Vivaswan Tech | Software Technologies Private Limited*, 15 May 2024, vivaswantech.com/service/web-and-mobile-app-development/.
- [6] FAQ. (n.d.). Retrieved from <https://docs.flutter.dev/resources/faq>
- [7] “Flutter_tts: Flutter Package.” *Dart Packages*, 14 Nov. 2024, pub.dev/packages/flutter_tts.
- [8] “GitHub Copilot · Your AI Pair Programmer.” GitHub, github.com/features/copilot.
- [9] Hinsenkamp, F. (2024). What is a Client Application? Retrieved from <https://www.bigtech.coach/system-components/what-is-a-client-application>
- [10] “Http: Dart Package.” *Dart Packages*, 16 July 2024, pub.dev/packages/http.
- [11] Ibm. (2024). What Is Mobile Application Development? Retrieved from <https://www.ibm.com/topics/mobile-application-development>
- [12] Ibm. “What Is a REST API?” *IBM*, 19 Dec. 2024, www.ibm.com/think/topics/rest-apis
- [13] Novotny, J. “How to Use cURL with RESTful APIs.” *Linode Guides & Tutorials*, Linode, 1 July 2022, www.linode.com/docs/guides/curl-for-rest-api/.
- [14] OpenAI. (2025). *ChatGPT* [Large language model]. <https://chatgpt.com>
- [15] “Postgres: Dart Package.” *Dart Packages*, 22 Dec. 2024, pub.dev/packages/postgres.
- [16] Sonaike, T. “App Development for iOS and Android: Top Strategies and Tools.” *The PickFu Blog*, 27 Apr. 2024, www.pickfu.com/blog/app-development-ios-and-android/.

- [17] SQL DDL to ERD Diagram. (n.d.). <https://www.devtoolsdaily.com/sql/ddl-to-diagram/>.
- [18] “Table_calendar: Flutter Package.” *Dart Packages*, 12 Dec. 2024, pub.dev/packages/table_calendar.
- [19] The “MVP First” Mindset Behind Every Successful Salesforce App - Aquiva Labs. <https://aquivalabs.com/blog/the-mvp-first-mindset-behind-every-successful-salesforce-app>
- [20] “The Role of AI in Modern Education.” *University of Iowa*, The University of Iowa, 27 Aug. 2024, onlineprograms.education.uiowa.edu/blog/role-of-ai-in-modern-education.
- [21] “Visual Studio Code.” *Flutter*, docs.flutter.dev/tools/vs-code.
- [22] “What Is Boilerplate Code?” *Stack Overflow*, 1 Nov. 1955, stackoverflow.com/questions/3992199/what-is-boilerplate-code.
- [23] What is a Container? (2024). Retrieved from <https://www.docker.com/resources/what-container/>

7. Appendices

The entire repository for the code used to create this app can be found at <https://github.com/norah-kuduk/fitness-app>. Code (including highlights and line numbers) was formatted for this paper using <http://hilite.me>.

7.1 Backend Server Requests

The code located in **server.dart**, which is used to create API endpoints to access data store in the database.

```
1 // imports
2 import 'dart:convert';
3 import 'dart:io';
4 import 'package:shelf/shelf.dart';
5 import 'package:shelf/shelf_io.dart' as io;
6 import 'package:shelf_router/shelf_router.dart';
7 import 'package:shelf_cors_headers/shelf_cors_headers.dart';
8 import 'package:my_backend/database_helper.dart';
9
10 // define headers to allow cross-origin requests
11 // necessary for Flutter to communicate with server
12 final _headers = {
```

```

13  'Access-Control-Allow-Origin': '*',
14  'Access-Control-Allow-Headers': 'Origin, Content-Type',
15  'Access-Control-Allow-Methods': 'GET, POST, PUT, DELETE, OPTIONS'
16 };
17
18 // entry point of the server application.
19 Future<void> main() async {
20   final dbHelper = DatabaseHelper();
21   await dbHelper.openConnection();
22   await dbHelper.createTables();
23
24   // router setup
25   final router = Router();
26
27   // define routes for handling HTTP requests
28   // each route corresponds to a specific CRUD operation on the database
29
30   // get all routines
31   router.get('/routine', (Request request) async {
32     final result = await dbHelper.connection.query('SELECT * FROM Routine');
33     final routines = result
34       .map((row) =>
35         {'RoutineID': row[0], 'RoutineName': row[1], 'Description': row[2]})
36       .toList();
37     return Response.ok(json.encode(routines),
38       headers: {..._headers, 'Content-Type': 'application/json'});
39   });
40
41   // insert a new routine
42   router.post('/routine', (Request request) async {
43     final payload = json.decode(await request.readAsString());
44     await dbHelper.connection.execute(
45       'INSERT INTO Routine (RoutineName, Description) VALUES (@name, @description)',
46       substitutionValues: {
47         'name': payload['RoutineName'],
48         'description': payload['Description'],
49       },
50     );
51     return Response.ok('Routine added', headers: _headers);
52   });
53
54   // update a routine
55   router.put('/routine/<id>', (Request request) async {
56     final id = int.parse(request.params['id']!);
57     final payload = json.decode(await request.readAsString());
58     await dbHelper.connection.execute(
59       'UPDATE Routine SET RoutineName = @name, Description = @description WHERE RoutineID = @id',
60       substitutionValues: {
61         'id': id,
62         'name': payload['RoutineName'],
63         'description': payload['Description'],
64       },

```

```

65     );
66     return Response.ok('Routine updated', headers: _headers);
67 });
68
69 // delete a routine
70 router.delete('/routine/<id>', (Request request) async {
71     final id = int.parse(request.params['id']!);
72     await dbHelper.connection.execute(
73         'DELETE FROM Routine WHERE RoutineID = @id',
74         substitutionValues: {
75             'id': id,
76         },
77     );
78     return Response.ok('Routine deleted', headers: _headers);
79 });
80
81
82 // get all exercises in DB
83 router.get('/exercise', (Request request) async {
84     final result = await dbHelper.connection
85         .query('SELECT * FROM Exercise ORDER BY ExerciseName ASC');
86     final exercises = result
87         .map((row) => {
88             'ExerciseID': row[0],
89             'ExerciseName': row[1],
90             'Description': row[2],
91         })
92         .toList();
93     return Response.ok(json.encode(exercises),
94         headers: {..._headers, 'Content-Type': 'application/json'});
95 });
96
97 // add a new exercise
98 router.post('/exercise', (Request request) async {
99     final payload = json.decode(await request.readAsString());
100    await dbHelper.connection.query(
101        'INSERT INTO Exercise (ExerciseName, Description) VALUES (@name, @description)',
102        substitutionValues: {
103            'name': payload['ExerciseName'],
104            'description': payload['Description'],
105        },
106    );
107    return Response.ok('Exercise added', headers: _headers);
108 });
109
110 // update an exercise
111 router.put('/exercise/<id>', (Request request) async {
112     final id = int.parse(request.params['id']!);
113     final payload = json.decode(await request.readAsString());
114     await dbHelper.connection.execute(
115         'UPDATE Exercise SET ExerciseName = @name, Description = @description WHERE ExerciseID =
116         substitutionValues: {

```

```

117         'id': id,
118         'name': payload['ExerciseName'],
119         'description': payload['Description'],
120     },
121 );
122     return Response.ok('Exercise updated', headers: _headers);
123 });
124
125 // delete an exercise
126 router.delete('/exercise/<id>', (Request request) async {
127     final id = int.parse(request.params['id']!);
128     await dbHelper.connection.execute(
129         'DELETE FROM Exercise WHERE ExerciseID = @id',
130         substitutionValues: {
131             'id': id,
132         },
133     );
134     return Response.ok('Routine deleted', headers: _headers);
135 });
136
137 // get exercises for a routine
138 router.get('/routine/<id>/exercise', (Request request) async {
139     final id = int.parse(request.params['id']!);
140     final result = await dbHelper.connection.query(
141         'SELECT * FROM RoutineExercise WHERE RoutineID = @id ORDER BY Ord ASC',
142         substitutionValues: {'id': id});
143     final routineExercises = result
144         .map((row) => {
145             'RoutineID': row[0],
146             'ExerciseID': row[1],
147             'Sets': row[2],
148             'Reps': row[3],
149             'HoldTime': row[4],
150             'Ord': row[5],
151             'Notes': row[6]
152         })
153         .toList();
154     return Response.ok(json.encode(routineExercises),
155         headers: {..._headers, 'Content-Type': 'application/json'});
156 });
157
158 // add exercises to an existing routine
159 router.post('/routine/<id>/exercise', (Request request) async {
160     final id = int.parse(request.params['id']!);
161     final payload = json.decode(await request.readAsString());
162     final exercises = payload['Exercises'] as List;
163
164     print('Received routine ID: $id');
165     print('Received exercises: $exercises');
166
167     for (final exercise in exercises) {
168         await dbHelper.connection.query(

```

```

169         'INSERT INTO RoutineExercise (RoutineID, ExerciseID, Sets, Reps, HoldTime, Notes, Ord)
170         substitutionValues: {
171             'routineID': id,
172             'exerciseID': exercise['ExerciseID'],
173             'sets': exercise['Sets'],
174             'reps': exercise['Reps'],
175             'holdTime': exercise['HoldTime'],
176             'notes': exercise['Notes'],
177             'ord': exercise['Ord'],
178         },
179     );
180 }
181
182 return Response.ok('Exercises added to routine', headers: _headers);
183 });
184
185 // update an exercise in a routine
186 router.put('/routine/<routineId>/exercise/<exerciseId>',
187     (Request request) async {
188         final routineId = int.parse(request.params['routineId']!);
189         final exerciseId = int.parse(request.params['exerciseId']!);
190         final payload = json.decode(await request.readAsString());
191
192         await dbHelper.connection.query(
193             'UPDATE RoutineExercise SET Reps = @reps, HoldTime = @holdTime, Notes = @notes WHERE Rout
194             substitutionValues: {
195                 'sets': payload['Sets'],
196                 'reps': payload['Reps'],
197                 'holdTime': payload['HoldTime'],
198                 'notes': payload['Notes'],
199                 'ord': payload['Ord'], // added 'Ord' to the query
200                 'routineId': routineId,
201                 'exerciseId': exerciseId
202             },
203         );
204
205         return Response.ok('Exercise updated in routine', headers: _headers);
206     });
207
208 // update exercises in a routine
209 router.put('/routine/<routineId>/exercise', (Request request) async {
210     final routineId = int.parse(request.params['routineId']!);
211     final payload = json.decode(await request.readAsString());
212     final exercises = payload['Exercises'] as List;
213
214     for (final exercise in exercises) {
215         await dbHelper.connection.query(
216             'UPDATE RoutineExercise SET Sets = @sets, Reps = @reps, HoldTime = @holdTime, Notes = @
217             substitutionValues: {
218                 'sets': exercise['Sets'],
219                 'reps': exercise['Reps'],
220                 'holdTime': exercise['HoldTime'],

```

```

221         'notes': exercise['Notes'],
222         'ord': exercise['Ord'],
223         'routineId': routineId,
224         'exerciseId': exercise['ExerciseID']
225     },
226 );
227 }
228
229 return Response.ok('Exercises updated in routine', headers: _headers);
230 });
231
232 // delete an exercise from a routine
233 router.delete('/routine/<routineId>/exercise/<exerciseId>',
234   (Request request) async {
235     final routineId = int.parse(request.params['routineId']!);
236     final exerciseId = int.parse(request.params['exerciseId']!);
237
238     await dbHelper.connection.query(
239       'DELETE FROM RoutineExercise WHERE RoutineID = @routineId AND ExerciseID = @exerciseId',
240       substitutionValues: {'routineId': routineId, 'exerciseId': exerciseId},
241     );
242
243     return Response.ok('Exercise deleted from routine', headers: _headers);
244   });
245
246 // delete all exercises from a routine
247 router.delete('/routine/<id>/exercise', (Request request) async {
248   final id = int.parse(request.params['id']!);
249
250   await dbHelper.connection.query(
251     'DELETE FROM RoutineExercise WHERE RoutineID = @id',
252     substitutionValues: {'id': id},
253   );
254
255   return Response.ok('All exercises deleted from routine', headers: _headers);
256 });
257
258 // fetch routines scheduled for a specific date
259 router.get('/routine/schedule/<date>', (Request request, String date) async {
260   // parse the date from the URL
261   final parsedDate = DateTime.parse(date);
262
263   // query the database with proper parameter substitution
264   final result = await dbHelper.connection.query(
265     'SELECT * FROM ScheduledRoutine WHERE ScheduledDate = @ScheduledDate',
266     substitutionValues: {
267       'ScheduledDate': parsedDate
268         .toIso8601String()
269         .split('T')
270         .first, // Convert to valid ISO8601 string for DATE
271     },
272   );

```

```

273
274 // convert the result to a List of maps
275 final scheduledRoutines = result
276     .map((row) => {
277         'RoutineID': row[0],
278         'ScheduledDate':
279             (row[1] as DateTime).toIso8601String().split('T').first,
280         'CompletionStatus': row[2],
281         'CompletionDateTime': row[3],
282         'Notes': row[4]
283     })
284     .toList();
285
286 if (scheduledRoutines.isEmpty) {
287     return Response.notFound('No routines scheduled for $date');
288 }
289 // return JSON response
290 return Response.ok(jsonEncode(scheduledRoutines));
291 });
292
293 // schedule a routine
294 router.post('/routine/schedule', (Request request) async {
295     try {
296         final payload = json.decode(await request.readAsString());
297         final routineId = payload['RoutineID'];
298         final scheduledDate = payload['ScheduledDate'];
299
300         await dbHelper.connection.query(
301             '''INSERT INTO ScheduledRoutine (RoutineID, ScheduledDate)
302                 VALUES (@routineId, @scheduledDate)
303                 ON CONFLICT (RoutineID, ScheduledDate)
304                 DO NOTHING''',
305             substitutionValues: {
306                 'routineId': routineId,
307                 'scheduledDate': scheduledDate,
308             },
309         );
310
311         return Response.ok('Routine scheduled successfully');
312     } catch (e) {
313         return Response.internalServerError(
314             body: 'Failed to schedule routine: $e');
315     }
316 });
317
318 // update completion status of a scheduled routine
319 router.put('/routineschedule/update', (Request request) async {
320     try {
321         final payload = json.decode(await request.readAsString());
322         final routineId = payload['RoutineID'];
323         final scheduledDate = payload['ScheduledDate'];
324         final completionStatus = payload['CompletionStatus'];

```

```

325     final completionDateTime = payload['CompletionDateTime'];
326     final notes = payload['Notes'];
327
328     await dbHelper.connection.query(
329         '''UPDATE ScheduledRoutine
330             SET CompletionStatus = @completionStatus,
331                 CompletionDateTime = @completionDateTime,
332                 Notes = @notes
333             WHERE RoutineID = @routineId AND ScheduledDate = @scheduledDate''',
334         substitutionValues: {
335             'routineId': routineId,
336             'scheduledDate': scheduledDate,
337             'completionStatus': completionStatus,
338             'completionDateTime': completionDateTime,
339             'notes': notes,
340         },
341     );
342
343     return Response.ok('Routine status updated successfully');
344 } catch (e) {
345     return Response.internalServerError(body: 'Failed to update routine: $e');
346 }
347 });
348
349 // delete a scheduled routine
350 router.delete('/routineschedule/delete', (Request request) async {
351     try {
352         final payload = json.decode(await request.readAsString());
353         final routineId = payload['RoutineID'];
354         final scheduledDate = payload['ScheduledDate'];
355
356         await dbHelper.connection.query(
357             'DELETE FROM ScheduledRoutine WHERE RoutineID = @routineId AND ScheduledDate = @scheduledDate',
358             substitutionValues: {
359                 'routineId': routineId,
360                 'scheduledDate': scheduledDate,
361             },
362         );
363
364         return Response.ok('Routine deleted successfully');
365     } catch (e) {
366         return Response.internalServerError(body: 'Failed to delete routine: $e');
367     }
368 });
369
370 // sets up the middleware pipeline for logging requests and handling CORS,
371 // and then starts the server.
372
373 final handler = const Pipeline()
374     .addMiddleware(logRequests())
375     .addMiddleware(corsHeaders())
376     .addHandler(router.call);

```

```

377
378   final port = int.parse(Platform.environment['PORT'] ?? '8080');
379   final server = await io.serve(handler, 'localhost', port);
380
381   print('Server listening on port ${server.port}');
382 }
383

```

7.2 Database Helper

The code located in **database_helper.dart**, used to connect to the ‘pt_flutter’ database which holds all information to the app. This code is not secure, as user and password are hard coded into the file, this would be changed if the app was ever deployed.

```

1  import 'dart:io';
2
3  import 'package:postgres/postgres.dart';
4
5  class DatabaseHelper {
6    final connection = PostgreSQLConnection(
7      'localhost',
8      5432,
9      'pt_flutter',
10     username: 'postgres',
11     password: 'pgpass',
12   );
13
14   Future<void> openConnection() async {
15     await connection.open();
16   }
17
18   Future<void> createTables() async {
19     final file = File('./DDL.sql');
20     final contents = await file.readAsString();
21
22     // split the content by semicolon to get individual statements
23     final statements =
24       contents.split(';').where((statement) => statement.trim().isNotEmpty);
25
26     // execute each statement
27     for (var statement in statements) {
28       await connection.execute(statement.trim());
29     }
30   }
31
32   Future<void> closeConnection() async {
33     await connection.close();
34   }

```

35 }

7.3 DDL

The code located in **DDL.sql**, used to create the tables needed for the database and called by **database_helper**.

```

1 CREATE TYPE completion_status AS ENUM ('Scheduled', 'Completed', 'Missed');
2
3 CREATE TABLE IF NOT EXISTS Routine (
4   RoutineID INT PRIMARY KEY,
5   RoutineName VARCHAR(255) NOT NULL,
6   Description TEXT
7 );
8
9 CREATE TABLE IF NOT EXISTS Exercise (
10  ExerciseID INT PRIMARY KEY,
11  ExerciseName VARCHAR(255) NOT NULL,
12  Description TEXT
13 );
14
15 CREATE TABLE IF NOT EXISTS RoutineExercise (
16  RoutineID INT NOT NULL,
17  ExerciseID INT NOT NULL,
18  Sets INT,
19  Reps INT,
20  HoldTime INT,
21  Ord INT,
22  Notes TEXT,
23  FOREIGN KEY (RoutineID) REFERENCES Routine(RoutineID),
24  FOREIGN KEY (ExerciseID) REFERENCES Exercise(ExerciseID)
25 );
26
27 CREATE TABLE IF NOT EXISTS ScheduledRoutine (
28  RoutineID INT NOT NULL,
29  ScheduledDate DATE NOT NULL,
30  CompletionStatus completion_status DEFAULT 'Scheduled',
31  CompletionDateTime TIMESTAMP NULL,
32  Notes TEXT,
33  FOREIGN KEY (RoutineID) REFERENCES Routine(RoutineID),
34  UNIQUE (RoutineID, ScheduledDate)
35 );

```

7.4 Minimum Viable Product (MVP)

The code is located in `_mvp.dart`, which holds the initial MVP, with local storage and hard-coded routines.

```

1 import 'dart:convert';
2 import 'package:flutter/material.dart';
3
4 void main() {
5   runApp(const MyApp());
6 }
7
8 class MyApp extends StatelessWidget {
9   const MyApp({super.key});
10
11   // widget is the root of your application.
12   @override
13   Widget build(BuildContext context) {
14     return MaterialApp(
15       title: 'PT MVP',
16       theme: ThemeData(
17         colorScheme: ColorScheme.fromSeed(
18           seedColor: const Color.fromARGB(255, 113, 139, 209)),
19         useMaterial3: true,
20       ),
21       home: const MyHomePage(title: 'PT Routine Options'),
22       routes: {
23         '/page1': (context) => const Page1(title: 'Page 1'),
24         '/page2': (context) => const Page2(title: 'Page 2'),
25       },
26     );
27   }
28 }
29
30 class MyHomePage extends StatefulWidget {
31   const MyHomePage({super.key, required this.title});
32
33   final String title;
34
35   @override
36   State<MyHomePage> createState() => _MyHomePageState();
37 }
38
39 class _MyHomePageState extends State<MyHomePage> {
40   @override
41   Widget build(BuildContext context) {
42     return Scaffold(
43       appBar: AppBar(
44         backgroundColor: Theme.of(context).colorScheme.inversePrimary,

```

```

45     title: Text(widget.title),
46   ),
47   body: Center(
48     child: Column(
49       mainAxisAlignment: MainAxisAlignment.center,
50       children: <Widget>[
51         ElevatedButton(
52           onPressed: () {
53             Navigator.pushNamed(context, '/page1');
54           },
55           child: const Text('Ed\'s PT Routine'),
56         ),
57         ElevatedButton(
58           onPressed: () {
59             Navigator.pushNamed(context, '/page2');
60           },
61           child: const Text('Norah\'s PT Routine'),
62         ),
63       ],
64     ),
65   ),
66 );
67 }
68 }
69
70 class Page1 extends StatefulWidget {
71   const Page1({super.key, required this.title});
72
73   final String title;
74   final String exercisesJson = '''
75   [
76     {
77       "name": "Pelvic bridges",
78       "reps": 10,
79       "hold_time_secs": 5,
80       "completed": false
81     },
82     {
83       "name": "Pelvic tilts",
84       "reps": 10,
85       "hold_time_secs": 5,
86       "completed": false
87     },
88     {
89       "name": "Knee squeeze with ball between knees (bottom on ground)",
90       "reps": 10,
91       "hold_time_secs": 5,
92       "completed": false
93     },
94     {
95       "name": "Bridges with ball knee squeeze",
96       "reps": 10,

```

```

97     "hold_time_secs": 5,
98     "completed": false
99 },
100 {
101     "name": "Bridges with strap around knees (knees go out)",
102     "reps": 10,
103     "hold_time_secs": 1,
104     "completed": false
105 },
106 {
107     "name": "Bottom flat on ground, strap around knees (each knee going out)",
108     "reps": 10,
109     "hold_time_secs": 1,
110     "completed": false
111 },
112 {
113     "name": "Lay on each side and clam shell",
114     "reps": 10,
115     "hold_time_secs": 1,
116     "completed": false
117 },
118 {
119     "name": "Lay on side and side abductors",
120     "reps": 10,
121     "hold_time_secs": 1,
122     "completed": false
123 },
124 {
125     "name": "Lay on belly and back leg lifts",
126     "reps": 10,
127     "hold_time_secs": 1,
128     "completed": false
129 },
130 {
131     "name": "Standing side leg abductors with strap",
132     "reps": 10,
133     "hold_time_secs": 1,
134     "completed": false
135 },
136 {
137     "name": "Standing back leg abduction with strap",
138     "reps": 10,
139     "hold_time_secs": 1,
140     "completed": false
141 },
142 {
143     "name": "Hamstring stretch",
144     "reps": 3,
145     "hold_time_secs": 20,
146     "completed": false
147 },
148 {

```

```

149     "name": "Hanging hip flexor stretch",
150     "reps": 3,
151     "hold_time_secs": 20,
152     "completed": false
153   }
154 ]
155 ''';
156
157 @override
158 State<Page1> createState() => _Page1State();
159 }
160
161 class _Page1State extends State<Page1> {
162   late List<dynamic> exercises;
163
164   @override
165   void initState() {
166     super.initState();
167     exercises = jsonDecode(widget.exercisesJson);
168   }
169
170   @override
171   Widget build(BuildContext context) {
172     return Scaffold(
173       appBar: AppBar(
174         title: const Text('Ed\'s PT Routine'),
175       ),
176       body: Stack(children: <Widget>[
177         ListView.builder(
178           itemCount: exercises.length,
179           itemBuilder: (context, index) {
180             return CheckboxListTile(
181               title: Text(
182                 exercises[index]['name'],
183                 style: TextStyle(
184                   decoration: exercises[index]['completed']
185                     ? TextDecoration.lineThrough
186                     : TextDecoration.none,
187                 ),
188             ),
189             subtitle: Text(
190               'Reps: ${exercises[index]['reps']} \nHold time: ${exercises[index]['hold_time_secs']}',
191             value: exercises[index]['completed'],
192             onChanged: (bool? value) {
193               setState(() {
194                 exercises[index]['completed'] = value ?? false;
195               });
196             },
197           );
198         },
199       ),
200       Positioned(

```

```

201         bottom: 16,
202         left: 16,
203         child: ElevatedButton(
204             onPressed: () {
205                 Navigator.pop(context);
206             },
207             child: const Icon(Icons.arrow_back),
208         ),
209     ),
210 ]),
211 );
212 }
213 }
214
215 class Page2 extends StatefulWidget {
216     const Page2({super.key, required this.title});
217
218     // This widget is the home page of your application. It is stateful, meaning
219     // that it has a State object (defined below) that contains fields that affect
220     // how it looks.
221
222     // This class is the configuration for the state. It holds the values (in this
223     // case the title) provided by the parent (in this case the App widget) and
224     // used by the build method of the State. Fields in a Widget subclass are
225     // always marked "final".
226
227     final String title;
228
229     @override
230     State<Page2> createState() => _Page2State();
231 }
232
233 class _Page2State extends State<Page2> {
234     @override
235     Widget build(BuildContext context) {
236         return Scaffold(
237             appBar: AppBar(
238                 title: const Text('Page 2'),
239             ),
240             body: Stack(children: <Widget>[
241                 Positioned(
242                     bottom: 16,
243                     left: 16,
244                     child: ElevatedButton(
245                         onPressed: () {
246                             Navigator.pop(context);
247                         },
248                         child: const Icon(Icons.arrow_back),
249                     ),
250                 ),
251             ]),
252         );

```

```
253 }
254 }
```

7.5 Main

The code located in **main.dart**. This code starts the app, as well as gets the system theme as the default value.

```
1 import 'package:flutter/material.dart';
2 import 'home_screen.dart'; // Import the HomeScreen widget
3
4 void main() {
5   runApp(const MyApp());
6 }
7
8 class MyApp extends StatefulWidget {
9   const MyApp({super.key});
10
11   @override
12   _MyAppState createState() => _MyAppState();
13 }
14
15 class _MyAppState extends State<MyApp> {
16   ThemeMode _themeMode = ThemeMode.system; // Default to system theme
17
18   void toggleTheme(bool isOn) {
19     setState(() {
20       _themeMode = isOn ? ThemeMode.dark : ThemeMode.light;
21     });
22   }
23
24   @override
25   Widget build(BuildContext context) {
26     return MaterialApp(
27       title: 'Flutter Demo',
28       theme: ThemeData(
29         primarySwatch: Colors.blue,
30         brightness: Brightness.light,
31       ),
32       darkTheme: ThemeData(
33         primarySwatch: Colors.blue,
34         brightness: Brightness.dark,
35       ),
36       themeMode: _themeMode, // Use the current theme mode
37       home: HomeScreen(
38         title: 'Routine List',
39         onToggleTheme:
40           toggleTheme, // Pass the toggle function to the HomeScreen
41     ),
```

```

42     );
43   }
44 }

```

7.6 Home Screen

The **home_screen.dart** file contains all of the details to view, update, and delete routines. It also holds the navigation to other screens within the app.

```

1  import 'package:flutter/material.dart';
2  import 'package:http/http.dart' as http;
3  import 'dart:convert';
4  import 'exercises_list_screen.dart';
5  import 'routine_info.dart';
6  import 'settings_screen.dart';
7  import 'calendar_screen.dart';
8
9  class HomeScreen extends StatefulWidget {
10    final String title;
11    final Function(bool) onToggleTheme;
12
13    const HomeScreen({super.key, required this.title, required this.onToggleTheme});
14
15    @override
16    _HomeScreenState createState() => _HomeScreenState();
17  }
18
19  class _HomeScreenState extends State<HomeScreen> {
20    List<dynamic> _routines = [];
21    final TextEditingController _nameController = TextEditingController();
22    final TextEditingController _descriptionController = TextEditingController();
23    bool _showDeleteOptions = false;
24    bool _showEditOptions = false;
25
26    @override
27    void initState() {
28      super.initState();
29      fetchRoutines();
30    }
31
32    Future<void> fetchRoutines() async {
33      try {
34        final response =
35          await http.get(Uri.parse('http://localhost:8080/routine'));
36
37        if (response.statusCode == 200) {
38          setState(() {
39

```

```

40         _routines = json.decode(response.body);
41     });
42     } else {
43         throw Exception('Failed to load routines');
44     }
45     } catch (e) {
46         // user message
47         ScaffoldMessenger.of(context).showSnackBar(
48             const SnackBar(
49                 content: Text('Failed to load routines'),
50             ),
51         );
52     }
53 }
54
55 Future<void> addRoutine(String name, String description) async {
56     if (name.isEmpty) {
57         throw Exception('Name cannot be empty');
58     }
59
60     final response = await http.post(
61         Uri.parse('http://localhost:8080/routine'),
62         headers: {'Content-Type': 'application/json'},
63         body: json.encode({'RoutineName': name, 'Description': description}),
64     );
65
66     if (response.statusCode == 200) {
67         fetchRoutines();
68     } else {
69         throw Exception('Failed to add routine');
70     }
71 }
72
73 Future<void> updateRoutine(int id, String name, String description) async {
74     final response = await http.put(
75         Uri.parse('http://localhost:8080/routine/$id'),
76         headers: {'Content-Type': 'application/json'},
77         body: json.encode({'RoutineName': name, 'Description': description}),
78     );
79
80     if (response.statusCode == 200) {
81         fetchRoutines();
82     } else {
83         throw Exception('Failed to update routine');
84     }
85 }
86
87 Future<void> deleteRoutine(int id) async {
88     final exerciseResponse = await http.get(
89         Uri.parse('http://localhost:8080/routine/$id/exercise'),
90     );
91 }

```

```

92     if (exerciseResponse.statusCode == 200) {
93         final exercises = json.decode(exerciseResponse.body);
94         if (exercises.isNotEmpty) {
95             bool confirmDelete = await _showDeleteConfirmationDialog();
96             if (!confirmDelete) {
97                 return;
98             } else {
99                 await http.delete(
100                     Uri.parse('http://localhost:8080/routine/$id/exercise'),
101                 );
102             }
103         }
104     } else {
105         throw Exception('Failed to check exercises for routine');
106     }
107
108     final response = await http.delete(
109         Uri.parse('http://localhost:8080/routine/$id'),
110     );
111
112     if (response.statusCode == 200) {
113         fetchRoutines();
114     } else {
115         throw Exception('Failed to delete routine');
116     }
117 }
118
119 Future<bool> _showDeleteConfirmationDialog() async {
120     return await showDialog<bool>(
121         context: context,
122         builder: (context) {
123             return AlertDialog(
124                 title: const Text('Confirm Delete'),
125                 content: const Text(
126                     'This routine has exercises. Are you sure you want to delete it?'),
127                 actions: [
128                     TextButton(
129                         onPressed: () {
130                             Navigator.of(context).pop(false);
131                         },
132                         child: const Text('Cancel'),
133                     ),
134                     TextButton(
135                         onPressed: () {
136                             Navigator.of(context).pop(true);
137                         },
138                         child: const Text('Delete'),
139                     ),
140                 ],
141             );
142         },
143     ) ??

```

```

144         false;
145     }
146
147     void _showRoutineEditDialog(
148         {int? id, String? currentName, String? currentDescription}) {
149         _nameController.text = currentName ?? '';
150         _descriptionController.text = currentDescription ?? '';
151         showDialog(
152             context: context,
153             builder: (context) {
154                 return AlertDialog(
155                     title: Text(id == null ? 'Add Routine' : 'Update Routine'),
156                     content: Column(
157                         mainAxisAlignment: MainAxisAlignment.min,
158                         children: [
159                             TextField(
160                                 controller: _nameController,
161                                 decoration: const InputDecoration(labelText: 'Routine Name'),
162                             ),
163                             TextField(
164                                 controller: _descriptionController,
165                                 decoration: const InputDecoration(labelText: 'Description'),
166                             ),
167                         ],
168                     ),
169                     actions: [
170                         TextButton(
171                             onPressed: () {
172                                 if (id == null) {
173                                     addRoutine(_nameController.text.toTitleCase(),
174                                         _descriptionController.text);
175                                 } else {
176                                     updateRoutine(id, _nameController.text.toTitleCase(),
177                                         _descriptionController.text);
178                                 }
179                                 Navigator.of(context).pop();
180                             },
181                             child: Text(id == null ? 'Add' : 'Update'),
182                         ),
183                         TextButton(
184                             onPressed: () => Navigator.of(context).pop(),
185                             child: const Text('Cancel'),
186                         ),
187                     ],
188                 );
189             },
190         );
191     }
192
193     @override
194     Widget build(BuildContext context) {
195         return Scaffold(

```

```

196     appBar: AppBar(
197       backgroundColor: Theme.of(context).colorScheme.inversePrimary,
198       title: Text(widget.title),
199     ),
200     drawer: Drawer(
201       child: ListView(
202         padding: EdgeInsets.zero,
203         children: <Widget>[
204           const DrawerHeader(
205             decoration: BoxDecoration(),
206             child: Text(
207               'Pages',
208               style: TextStyle(
209                 color: Color.fromARGB(255, 75, 91, 139),
210                 fontSize: 24,
211             ),
212           ),
213         ],
214         ListTile(
215           leading: const Icon(Icons.sports_gymnastics),
216           title: const Text('Exercises'),
217           onTap: () {
218             Navigator.of(context).push(MaterialPageRoute(
219               builder: (context) => const ExerciseScreen(),
220             ));
221           },
222         ),
223         ListTile(
224           leading: const Icon(Icons.settings),
225           title: const Text('Settings'),
226           onTap: () {
227             Navigator.of(context).push(
228               MaterialPageRoute(
229                 builder: (context) => SettingsScreen(
230                   onToggleTheme: widget.onToggleTheme,
231                   isDarkMode:
232                     Theme.of(context).brightness == Brightness.dark,
233                 ),
234             ),
235           );
236         },
237       ),
238       ListTile(
239         leading: const Icon(Icons.calendar_month),
240         title: const Text('Calendar'),
241         onTap: () {
242           Navigator.of(context).push(MaterialPageRoute(
243             builder: (context) => const CalendarScreen(),
244           ));
245         },
246       ),
247     ],

```

```

248     ),
249   ),
250   body: Column(children: [
251     Expanded(
252       child: ListView.builder(
253         itemCount: _routines.length,
254         itemBuilder: (context, index) {
255           final routine = _routines[index];
256           return ListTile(
257             title: Text(routine['RoutineName']),
258             onTap: () {
259               Navigator.push(
260                 context,
261                 MaterialPageRoute(
262                   builder: (context) => RoutineInfoScreen(
263                     routineId: routine['RoutineID'],
264                     routineName: routine['RoutineName'],
265                   ),
266                 ),
267             );
268           },
269           trailing: Row(
270             mainAxisAlignment:
271               MainAxisAlignment.min, // Important to avoid overflow
272             children: <Widget>[
273               if (_showDeleteOptions)
274                 IconButton(
275                   icon: const Icon(Icons.delete, color: Colors.red),
276                   onPressed: () {
277                     deleteRoutine(routine['RoutineID']);
278                   },
279                 ),
280               if (_showEditOptions)
281                 IconButton(
282                   icon: const Icon(Icons.edit,
283                     color: Color.fromARGB(255, 81, 81, 81)),
284                   onPressed: () {
285                     _showRoutineEditDialog(
286                       id: routine['RoutineID'],
287                       currentName: routine['RoutineName'],
288                       currentDescription: routine['Description']);
289                   },
290                 ),
291             ]));
292     },
293   ),
294 ),
295 Padding(
296   padding: const EdgeInsets.all(16.0),
297   child: Row(
298     children: [
299       ElevatedButton(

```

```

300         onPressed: _showRoutineEditDialog,
301         child: const Icon(Icons.add),
302     ),
303     const SizedBox(width: 8),
304     ElevatedButton(
305         onPressed: () {
306             setState(() {
307                 _showDeleteOptions = !_showDeleteOptions;
308             });
309         },
310         child: Icon(_showDeleteOptions ? Icons.check : Icons.delete),
311     ),
312     const SizedBox(width: 8),
313     ElevatedButton(
314         onPressed: () {
315             setState(() {
316                 _showEditOptions = !_showEditOptions;
317             });
318         },
319         child: Icon(_showEditOptions ? Icons.check : Icons.edit),
320     ),
321 ],
322 ),
323 ),
324 ]),
325 );
326 }

```

7.7 Routine Info Screen

The **routine_info.dart** file contains all of the information to view the exercises in a routine, as well as add and edit the exercises. It also has the code to schedule a routine and have it display in the calendar.

```

1  // this create/display the information about each routine and exercise
2  // have capabilities to add, update, and delete exercises, reps, sets,
3  // hold time, equipment, notes etc.
4
5  import 'package:flutter/material.dart';
6  import 'package:http/http.dart' as http;
7  import 'dart:convert';
8  import 'schedule_dialog.dart';
9  import 'package:flutter_tts/flutter_tts.dart';
10
11 class RoutineInfoScreen extends StatefulWidget {
12     final int routineId;
13     final String routineName;
14

```

```

15     const RoutineInfoScreen(
16         {super.key, required this.routineId, required this.routineName});
17
18     @override
19     _RoutineInfoScreenState createState() => _RoutineInfoScreenState();
20 }
21
22 class _RoutineInfoScreenState extends State<RoutineInfoScreen> {
23     List<dynamic> _exercises = [];
24     List<dynamic> _routineExercises = [];
25     List<Map<String, dynamic>> _selectedExercises = [];
26     List<dynamic> _filteredExercises = [];
27     final TextEditingController _searchController = TextEditingController();
28     bool _showDeleteOptions = false;
29     bool _showEditOptions = false;
30     late FlutterTts _flutterTts; // Declare FlutterTts
31     bool _isPlaying = false; // Track if TTS is playing
32     String _lastSpokenText = ""; // Track last spoken text for resume
33
34     @override
35     void initState() {
36         super.initState(); // initialize
37         _flutterTts = FlutterTts(); // Initialize FlutterTts
38
39         fetchExercises(); // fetch exercises
40         fetchRoutineExercises(); // fetch routine exercises
41         _searchController
42             .addListener(_filterExercises); // listen to search controller
43     }
44
45     @override
46     void dispose() {
47         _searchController.dispose();
48         _flutterTts.stop(); // Stop any ongoing speech when the widget is disposed
49         super.dispose();
50     }
51
52     // fetch exercises from server
53     Future<void> fetchExercises() async {
54         final response =
55             await http.get(Uri.parse('http://localhost:8080/exercise'));
56
57         if (response.statusCode == 200) {
58             setState(() {
59                 _exercises = json.decode(response.body);
60             });
61         } else {
62             throw Exception('Failed to load exercises');
63         }
64     }
65
66     // fetch routine exercises from server

```

```

67 Future<void> fetchRoutineExercises() async {
68   try {
69     final response = await http.get(Uri.parse(
70       'http://localhost:8080/routine/${widget.routineId}/exercise'));
71
72     if (response.statusCode == 200) {
73       setState(() {
74         _routineExercises = json.decode(response.body);
75       });
76     } else {
77       throw Exception('Failed to load routine exercises');
78     }
79   } catch (e) {
80     // user message
81     ScaffoldMessenger.of(context).showSnackBar(
82       const SnackBar(
83         content: Text('Failed to load exercises'),
84       ),
85     );
86   }
87 }
88
89 // Read out the exercise details using Text-to-Speech
90 Future<void> _speakRoutineDetails() async {
91   String textToRead = 'Routine: ${widget.routineName}. ';
92   if (_routineExercises.isNotEmpty) {
93     for (var exercise in _routineExercises) {
94       textToRead +=
95         'Exercise: ${_exercises.firstWhere((e) => e['ExerciseID'] == exercise
96         textToRead += '${exercise['Sets']} sets of ${exercise['Reps']} reps.';
97         if (exercise['HoldTime'] != null && exercise['HoldTime'] > 0) {
98           textToRead += 'Hold for ${exercise['HoldTime']} seconds. ';
99         }
100         if (exercise['Notes'] != null && exercise['Notes'].isNotEmpty) {
101           textToRead += 'Notes: ${exercise['Notes']}. ';
102         }
103       }
104     } else {
105       textToRead += 'There are no exercises in this routine.';
106     }
107
108     await _flutterTts.setLanguage("en-US"); // Set language
109     await _flutterTts.setPitch(1.0); // Set pitch (0.5 to 2.0)
110     await _flutterTts.setSpeechRate(0.5); // Set speech rate (0.0 to 1.0)
111
112     setState(() {
113       _isPlaying = true;
114       _lastSpokenText = textToRead; // Track the last spoken text
115     });
116
117     await _flutterTts.speak(textToRead); // Speak the constructed text
118   }

```

```

119
120 // Pause the TTS
121 Future<void> _pauseTts() async {
122   if (_isPlaying) {
123     await _flutterTts.stop();
124     setState(() {
125       _isPlaying = false; // Track that TTS is paused
126     });
127   }
128 }
129
130 // Resume the TTS
131 Future<void> _resumeTts() async {
132   if (!_isPlaying && _lastSpokenText.isNotEmpty) {
133     setState(() {
134       _isPlaying = true; // Mark TTS as playing again
135     });
136     await _flutterTts.speak(_lastSpokenText); // Speak the last text again
137   }
138 }
139
140 // add exercises to routine
141 Future<void> addExercisesToRoutine() async {
142   // Filter out exercises that are already in the routine
143   List exercisesToAdd = _selectedExercises
144     .where((exercise) => !_routineExercises.contains(exercise['id']))
145     .toList();
146
147   // Proceed only if there are new exercises to add
148   if (exercisesToAdd.isNotEmpty) {
149     final response = await http.post(
150       Uri.parse('http://localhost:8080/routine/${widget.routineId}/exercise'),
151       headers: {'Content-Type': 'application/json'},
152       body: json.encode({'Exercises': exercisesToAdd}),
153     );
154
155     if (response.statusCode == 200) {
156       fetchRoutineExercises();
157     } else {
158       throw Exception('Failed to add exercises to routine');
159     }
160   } else {
161     print('No new exercises to add');
162     // Optionally, handle the case when there are no new exercises to add
163   }
164 }
165
166 // update routine exercise on server
167 Future<void> updateRoutineExercise(
168   int exerciseId, int sets, int reps, int holdTime, String notes) async {
169   final response = await http.put(
170     Uri.parse(

```

```

171         'http://localhost:8080/routine/${widget.routineId}/exercise/${exerciseId}
172         headers: {'Content-Type': 'application/json'},
173         body: json.encode(
174             {'Sets': sets, 'Reps': reps, 'HoldTime': holdTime, 'Notes': notes}),
175     );
176
177     if (response.statusCode == 200) {
178         fetchRoutineExercises();
179     } else {
180         throw Exception('Failed to update exercise in routine');
181     }
182 }
183
184 // update all server routine exercises with order
185 Future<void> updateExercisesOnServer() async {
186     // first delete all current routine exercises
187     await http.delete(
188         Uri.parse('http://localhost:8080/routine/${widget.routineId}/exercise'),
189     );
190
191     // then add all routine exercises in the new order
192     if (_routineExercises.isNotEmpty) {
193         final response = await http.post(
194             Uri.parse('http://localhost:8080/routine/${widget.routineId}/exercise'),
195             headers: {'Content-Type': 'application/json'},
196             body: jsonEncode({
197                 'Exercises': _routineExercises,
198             })),
199     );
200
201     if (response.statusCode == 200) {
202         print('Server updated successfully.');
```

```

223     }
224
225     // filter exercises given a search query
226     void _filterExercises() {
227         final query = _searchController.text.toLowerCase();
228         setState(() {
229             _filteredExercises = _exercises.where((exercise) {
230                 final exerciseName = exercise['ExerciseName'].toLowerCase();
231                 return exerciseName.contains(query);
232             }).toList();
233         });
234     }
235
236     // open schedule dialog
237     void _openScheduleDialog(BuildContext context) {
238         showDialog(
239             context: context,
240             builder: (context) {
241                 return ScheduleDialog(routineId: widget.routineId);
242             },
243         );
244     }
245
246     // code for selecting exercises to add to routine
247     void _showExerciseSelectionDialog() {
248         List<Map<String, dynamic>> selectedExercises =
249             List.from(_selectedExercises);
250
251         showDialog(
252             context: context,
253             builder: (context) {
254                 return StatefulBuilder(
255                     builder: (context, setState) {
256                         _searchController.addListener(() {
257                             setState(() {
258                                 _filteredExercises = _exercises.where((exercise) {
259                                     final exerciseName = exercise['ExerciseName'].toLowerCase();
260                                     return exerciseName
261                                         .contains(_searchController.text.toLowerCase());
262                                 }).toList();
263                             });
264                         });
265                 return AlertDialog(
266                     title: const Text('Select Exercises'),
267                     content: Column(
268                         mainAxisAlignment: MainAxisAlignment.min,
269                         children: [
270                             TextField(
271                                 controller: _searchController,
272                                 decoration: const InputDecoration(
273                                     labelText: 'Search',
274                                     suffixIcon: Icon(Icons.search),

```

```

275         ),
276     ),
277     Expanded(
278       child: SizedBox(
279         width: double.maxFinite,
280         child: ListView.builder(
281           shrinkWrap: true,
282           itemCount: _filteredExercises.length,
283           itemBuilder: (context, index) {
284             final exercise = _filteredExercises[index];
285             return CheckboxListTile(
286               title: Text(exercise['ExerciseName']),
287               value: selectedExercises.any((e) =>
288                 e['ExerciseID'] == exercise['ExerciseID']),
289               onChanged: (bool? value) {
290                 setState(() {
291                   if (value == true) {
292                     selectedExercises.add({
293                       'ExerciseID': exercise['ExerciseID'],
294                       'Sets': 3, // Default value, can be modified
295                       'Reps':
296                         10, // Default value, can be modified
297                       'HoldTime':
298                         30, // Default value, can be modified
299                       'Notes':
300                         '' // Default value, can be modified
301                     });
302                   } else {
303                     selectedExercises.removeWhere((e) =>
304                       e['ExerciseID'] ==
305                       exercise['ExerciseID']);
306                   }
307                 });
308               },
309             );
310           },
311         ),
312       ),
313     ),
314   ],
315 ),
316 actions: [
317   TextButton(
318     onPressed: () {
319       Navigator.of(context).pop();
320       setState(() {
321         _selectedExercises = selectedExercises;
322       });
323       addExercisesToRoutine();
324     },
325     child: const Text('Add'),
326   ),

```

```

327         TextButton(
328             onPressed: () {
329                 Navigator.of(context).pop();
330             },
331             child: const Text('Cancel'),
332         ),
333     ],
334 );
335 },
336 );
337 },
338 );
339 }
340
341 // code for reordering exercises
342 void _onReorder(int oldIndex, int newIndex) {
343     setState(() {
344         if (newIndex > oldIndex) {
345             newIndex -= 1;
346         }
347         final item = _routineExercises.removeAt(oldIndex);
348         _routineExercises.insert(newIndex, item);
349
350         // Update the order property for each exercise
351         for (int i = 0; i < _routineExercises.length; i++) {
352             _routineExercises[i]['Ord'] = i;
353         }
354
355         // Optionally, save the new order here, either locally or by sending to a b
356         updateExercisesOnServer();
357     });
358 }
359
360 // code for editing an exercise
361 void _showEditExerciseDialog(Map<String, dynamic> exercise) {
362     final TextEditingController setsController =
363         TextEditingController(text: exercise['Sets'].toString());
364     final TextEditingController repsController =
365         TextEditingController(text: exercise['Reps'].toString());
366     final TextEditingController holdTimeController =
367         TextEditingController(text: exercise['HoldTime'].toString());
368     final TextEditingController notesController =
369         TextEditingController(text: exercise['Notes'].toString());
370
371     showDialog(
372         context: context,
373         builder: (context) {
374             return AlertDialog(
375                 title: const Text('Edit Exercise'),
376                 content: Column(
377                     mainAxisAlignment: MainAxisAlignment.min,
378                     children: [

```

```

379         TextField(
380             controller: setsController,
381             decoration: const InputDecoration(labelText: 'Sets'),
382             keyboardType: TextInputType.number,
383         ),
384         TextField(
385             controller: repsController,
386             decoration: const InputDecoration(labelText: 'Reps'),
387             keyboardType: TextInputType.number,
388         ),
389         TextField(
390             controller: holdTimeController,
391             decoration:
392                 const InputDecoration(labelText: 'Hold Time (seconds)'),
393             keyboardType: TextInputType.number,
394         ),
395         TextField(
396             controller: notesController,
397             decoration: const InputDecoration(labelText: 'Notes'),
398         ),
399     ],
400 ),
401     actions: [
402         TextButton(
403             onPressed: () {
404                 updateRoutineExercise(
405                     exercise['ExerciseID'],
406                     int.parse(setsController.text),
407                     int.parse(repsController.text),
408                     int.parse(holdTimeController.text),
409                     notesController.text,
410                 );
411                 Navigator.of(context).pop();
412             },
413             child: const Text('Save'),
414         ),
415         TextButton(
416             onPressed: () {
417                 Navigator.of(context).pop();
418             },
419             child: const Text('Cancel'),
420         ),
421     ],
422 );
423 },
424 );
425 }
426
427 // build the routine info screen
428 @override
429 Widget build(BuildContext context) {
430     return Scaffold(

```

```

431     appBar: AppBar(
432       title: Text(widget.routineName),
433     ),
434     body: Column(
435       children: [
436         Expanded(
437           child: ReorderableListView.builder(
438             onReorder: _onReorder,
439             itemCount: _routineExercises.length,
440             itemBuilder: (context, index) {
441               final routineExercise = _routineExercises[index];
442               final exerciseName = _exercises.firstWhere(
443                 (e) => e['ExerciseID'] == routineExercise['ExerciseID'],
444                 orElse: () =>
445                   {'ExerciseName': 'Unknown Exercise'})['ExerciseName'];
446               return ListTile(
447                 key: Key('$index'),
448                 title: Text(exerciseName),
449                 subtitle: Column(
450                   crossAxisAlignment: CrossAxisAlignment.start,
451                   children: [
452                     Text(
453                       '${routineExercise['Sets']} x ${routineExercise['Reps']}
454                       Text('Hold Time: ${routineExercise['HoldTime']} sec'),
455                       Text('Notes: ${routineExercise['Notes']}'),
456                     ],
457                 ),
458                 trailing: Row(
459                   mainAxisAlignment: MainAxisAlignment.min,
460                   children: [
461                     if (_showEditOptions)
462                       IconButton(
463                         icon: const Icon(Icons.edit),
464                         onPressed: () {
465                           _showEditExerciseDialog(routineExercise);
466                         },
467                       ),
468                     if (_showDeleteOptions)
469                       IconButton(
470                         icon: const Icon(Icons.delete, color: Colors.red),
471                         onPressed: () {
472                           deleteRoutineExercise(
473                             routineExercise['ExerciseID']);
474                         },
475                       ),
476                   ],
477                 ),
478               );
479             },
480           ),
481         ),
482         Padding(

```

```

483         padding: const EdgeInsets.only(left: 16.0),
484         child: Row(
485           children: [
486             ElevatedButton(
487               onPressed: _showExerciseSelectionDialog,
488               child: const Icon(Icons.add),
489             ),
490             ElevatedButton(
491               onPressed: () {
492                 setState(() {
493                   _showDeleteOptions = !_showDeleteOptions;
494                 });
495               },
496               child: Icon(_showDeleteOptions ? Icons.check : Icons.delete),
497             ),
498             ElevatedButton(
499               onPressed: () {
500                 setState(() {
501                   _showEditOptions = !_showEditOptions;
502                 });
503               },
504               child: Icon(_showEditOptions ? Icons.check : Icons.edit),
505             ),
506             ElevatedButton(
507               onPressed: () => _openScheduleDialog(context),
508               child: const Icon(Icons.calendar_today),
509             ),
510           ],
511         ),
512       ),
513       Padding(
514         padding: const EdgeInsets.only(left: 16.0) +
515           const EdgeInsets.only(top: 8.0) +
516           const EdgeInsets.only(bottom: 8.0),
517         child: Row(
518           children: [
519             ElevatedButton(
520               onPressed: _speakRoutineDetails,
521               child: const Icon(Icons.volume_up),
522             ),
523             ElevatedButton(
524               onPressed: _pauseTts,
525               child: const Icon(Icons.pause),
526             ),
527             ElevatedButton(
528               onPressed: _resumeTts,
529               child: const Icon(Icons.play_arrow),
530             ),
531           ],
532         ),
533       ),
534     ],

```

```

535         ),
536     );
537 }
538 }

```

7.8 Exercise List Screen

The **exercise_list_screen.dart** file holds the code to display all exercises currently in the database.

```

1  // this will have the list of exercises with links to each of their pages
2  // exercise_screen.dart
3  import 'package:flutter/material.dart';
4  import 'package:http/http.dart' as http;
5  import 'dart:convert';
6
7  extension StringExtension on String {
8      String toTitleCase() {
9          return split(' ')
10             .map((word) => word.isNotEmpty
11                 ? '${word[0].toUpperCase()}${word.substring(1).toLowerCase()}'
12                 : word)
13             .join(' ');
14     }
15 }
16
17 class ExerciseScreen extends StatefulWidget {
18     const ExerciseScreen({super.key});
19
20     @override
21     _ExerciseScreenState createState() => _ExerciseScreenState();
22 }
23
24 class _ExerciseScreenState extends State<ExerciseScreen> {
25     List<dynamic> _exercises = [];
26     final TextEditingController _nameController = TextEditingController();
27     final TextEditingController _descriptionController = TextEditingController();
28     bool _showDeleteOptions = false;
29     bool _showEditOptions = false;
30
31     @override
32     void initState() {
33         super.initState();
34         fetchExercises();
35     }
36
37     Future<void> fetchExercises() async {
38

```

```

39     try {
40         final response =
41             await http.get(Uri.parse('http://localhost:8080/exercise'));
42
43         setState(() {
44             _exercises = json.decode(response.body);
45         });
46     } catch (e) {
47         // user message
48         ScaffoldMessenger.of(context).showSnackBar(
49             const SnackBar(
50                 content: Text('Failed to load exercises'),
51             ),
52         );
53     }
54 }
55
56 Future<void> addExercise(String name, String description) async {
57     if (name.isEmpty) {
58         throw Exception('Name cannot be empty');
59     }
60
61     final response = await http.post(
62         Uri.parse('http://localhost:8080/exercise'),
63         headers: {'Content-Type': 'application/json'},
64         body: json.encode({
65             'ExerciseName': name,
66             'Description': description,
67         })),
68     );
69
70     if (response.statusCode == 200) {
71         fetchExercises();
72     } else {
73         throw Exception('Failed to add exercise');
74     }
75 }
76
77 Future<void> updateExercise(int id, String name, String description) async {
78     final response = await http.put(
79         Uri.parse('http://localhost:8080/exercise/$id'),
80         headers: {'Content-Type': 'application/json'},
81         body: json.encode({'ExerciseName': name, 'Description': description}),
82     );
83
84     if (response.statusCode == 200) {
85         fetchExercises();
86     } else {
87         throw Exception('Failed to update exercise');
88     }
89 }
90

```

```

91 Future<void> deleteExercise(int id) async {
92     final response = await http.delete(
93         Uri.parse('http://localhost:8080/exercise/$id'),
94     );
95
96     if (response.statusCode == 200) {
97         fetchExercises();
98     } else {
99         throw Exception('Failed to delete exercise');
100     }
101 }
102
103 void _showExerciseEditDialog(
104     {int? id, String? currentName, String? currentDescription}) {
105     _nameController.text = currentName ?? '';
106     _descriptionController.text = currentDescription ?? '';
107     showDialog(
108         context: context,
109         builder: (context) {
110             return AlertDialog(
111                 title: Text(id == null ? 'Add Exercise' : 'Update Exercise'),
112                 content: Column(
113                     mainAxisAlignment: MainAxisAlignment.min,
114                     children: [
115                         TextField(
116                             controller: _nameController,
117                             decoration: const InputDecoration(labelText: 'Exercise Name'),
118                         ),
119                         TextField(
120                             controller: _descriptionController,
121                             decoration: const InputDecoration(labelText: 'Description'),
122                         ),
123                     ],
124                 ),
125                 actions: [
126                     TextButton(
127                         onPressed: () {
128                             if (id == null) {
129                                 addExercise(_nameController.text.titleCase(),
130                                     _descriptionController.text);
131                             } else {
132                                 updateExercise(id, _nameController.text.titleCase(),
133                                     _descriptionController.text);
134                             }
135                             Navigator.of(context).pop();
136                         },
137                         child: Text(id == null ? 'Add' : 'Update'),
138                     ),
139                     TextButton(
140                         onPressed: () => Navigator.of(context).pop(),
141                         child: const Text('Cancel'),
142                     ),
143                 ],
144             );
145         },
146     );
147 }

```

```

143     ],
144   );
145 },
146 );
147 }
148
149 @override
150 Widget build(BuildContext context) {
151   _exercises.sort((a, b) => a['ExerciseName'].compareTo(b['ExerciseName']));
152
153   return Scaffold(
154     appBar: AppBar(
155       title: const Text('Exercises'),
156     ),
157     body: Column(children: [
158       Expanded(
159         child: ListView.builder(
160           itemCount: _exercises.length,
161           itemBuilder: (context, index) {
162             final exercise = _exercises[index];
163             return ListTile(
164               title: Text(exercise['ExerciseName']),
165               // if exercise['Description'] is '' or null, use 'No description'
166               subtitle: Text(exercise['Description'] ?? 'No description'),
167               trailing: Row(
168                 mainAxisAlignment:
169                   MainAxisAlignment.min, // Important to avoid overflow
170                 children: <Widget>[
171                   if (_showDeleteOptions)
172                     IconButton(
173                       icon: const Icon(Icons.delete, color: Colors.red),
174                       onPressed: () {
175                         deleteExercise(exercise['ExerciseID']);
176                       },
177                     ),
178                   if (_showEditOptions)
179                     IconButton(
180                       icon: const Icon(Icons.edit,
181                         color: Color.fromARGB(255, 81, 81, 81)),
182                       onPressed: () {
183                         _showExerciseEditDialog(
184                           id: exercise['ExerciseID'],
185                           currentName: exercise['ExerciseName'],
186                           currentDescription: exercise['Description']);
187                       },
188                     ),
189                 ]));
190             },
191           )),
192     Padding(
193       padding: const EdgeInsets.all(16.0),
194       child: Row(

```

```

195         children: [
196           ElevatedButton(
197             onPressed: _showExerciseEditDialog,
198             child: const Icon(Icons.add),
199           ),
200           const SizedBox(width: 8),
201           ElevatedButton(
202             onPressed: () {
203               setState(() {
204                 _showDeleteOptions = !_showDeleteOptions;
205               });
206             },
207             child: Icon(_showDeleteOptions ? Icons.check : Icons.delete),
208           ),
209           const SizedBox(width: 8),
210           ElevatedButton(
211             onPressed: () {
212               setState(() {
213                 _showEditOptions = !_showEditOptions;
214               });
215             },
216             child: Icon(_showEditOptions ? Icons.check : Icons.edit),
217           ),
218         ],
219       ),
220     ),
221   ]),
222 );
223 }
224 }

```

7.9 Settings Screen

The **settings_screen.dart** file has code to display and change the app settings.

```

1 import 'package:flutter/material.dart';
2
3 class SettingsScreen extends StatefulWidget {
4   final bool isDarkMode;
5   final Function(bool) onToggleTheme;
6
7   const SettingsScreen(
8     {super.key, required this.onToggleTheme, required this.isDarkMode});
9
10  @override
11  _SettingsScreenState createState() => _SettingsScreenState();
12 }
13
14 class _SettingsScreenState extends State<SettingsScreen> {
15   @override
16   Widget build(BuildContext context) {
17     return Scaffold(

```

```

18     appBar: AppBar(
19       title: const Text('Settings'),
20     ),
21     body: ListView(
22       children: [
23         ListTile(
24           title: const Text('Toggle Dark Mode'),
25           trailing: Switch(
26             value: widget.isDarkMode,
27             onChanged: (bool value) {
28               widget.onToggleTheme(value);
29             },
30             activeThumbImage: const AssetImage('assets/moon.png'),
31             inactiveThumbImage: const AssetImage('assets/sun.png'),
32           ),
33         ],
34       ),
35     );
36   }
37 }

```

7.10 Calendar Screen

The **calendar_screen.dart** file has the code used to display the calendar page, with options to change the view type and focus on specific days.

```

1 import 'package:flutter/material.dart';
2 import 'package:table_calendar/table_calendar.dart';
3 import 'package:http/http.dart' as http;
4 import 'dart:convert';
5 import 'routine_info.dart';
6
7 class CalendarScreen extends StatefulWidget {
8   const CalendarScreen({super.key});
9
10  @override
11  _CalendarScreenState createState() => _CalendarScreenState();
12 }
13
14 class _CalendarScreenState extends State<CalendarScreen> {
15   DateTime _focusedDay = DateTime.now();
16   DateTime? _selectedDay;
17   List<dynamic> _routines = [];
18   List<dynamic> _availableRoutines = [];
19
20  @override
21  void initState() {
22    super.initState();
23    fetchRoutines();

```

```

24  }
25
26  // get all possible routines
27  Future<void> fetchRoutines() async {
28    try {
29      final response =
30        await http.get(Uri.parse('http://localhost:8080/routine'));
31
32      if (response.statusCode == 200) {
33        setState(() {
34          _availableRoutines = json.decode(response.body);
35        });
36      } else {
37        throw Exception('Failed to load routines');
38      }
39    } catch (e) {
40      // user message
41      ScaffoldMessenger.of(context).showSnackBar(
42        const SnackBar(
43          content: Text('Failed to load routines'),
44        ),
45      );
46    }
47  }
48
49  // fetch scheduled exercises for the selected day
50  Future<void> fetchScheduledRoutines() async {
51    try {
52      final url =
53        'http://localhost:8080/routine/schedule/${_selectedDay!.toIso8601String().split('T')}';
54      final response = await http.get(Uri.parse(url));
55
56      if (response.statusCode == 200) {
57        setState(() {
58          _routines = json.decode(response.body);
59        });
60      } else if (response.statusCode == 404) {
61        setState(() {
62          _routines = [];
63        });
64        throw Exception('No routines scheduled for the selected day');
65      }
66    } catch (e) {
67      print('Exception: $e');
68    }
69  }
70
71  void _openScheduleDialog(BuildContext context) async {
72    showDialog(
73      context: context,
74      builder: (BuildContext context) {
75        return AlertDialog(

```

```

76         title: const Text('Select a Routine to Schedule'),
77         content: _availableRoutines.isNotEmpty
78           ? SizedBox(
79             width: double.maxFinite,
80             child: ListView.builder(
81               itemCount: _availableRoutines.length,
82               itemBuilder: (context, index) {
83                 final routine = _availableRoutines[index];
84                 return ListTile(
85                   title: Text(routine['RoutineName']),
86                   onTap: () {
87                     _scheduleRoutineForDate(
88                       _selectedDay!, routine['RoutineID']);
89                     Navigator.pop(context);
90                   },
91                 );
92               },
93             ),
94           )
95         : const Text('No routines available.'),
96         actions: [
97           TextButton(
98             onPressed: () => Navigator.pop(context),
99             child: const Text('Cancel'),
100           ),
101         ],
102       );
103     },
104   );
105 }
106
107 Future<void> _scheduleRoutineForDate(DateTime date, int routineId) async {
108   try {
109     final response = await http.post(
110       Uri.parse('http://localhost:8080/routine/schedule'),
111       headers: {'Content-Type': 'application/json'},
112       body: json.encode({
113         'RoutineID': routineId,
114         // Convert the date to a string in the format 'YYYY-MM-DD'
115         'ScheduledDate': date.toIso8601String().split('T').first,
116       })),
117     );
118
119     if (response.statusCode == 200) {
120       fetchScheduledRoutines();
121       ScaffoldMessenger.of(context).showSnackBar(
122         const SnackBar(
123           content: Text('Routine scheduled successfully'),
124         ),
125       );
126     } else {
127       throw Exception('Failed to schedule routine');

```

```

128     }
129   } catch (e) {
130     ScaffoldMessenger.of(context).showSnackBar(
131       const SnackBar(
132         content: Text('Failed to schedule routine'),
133       ),
134     );
135   }
136 }
137
138 Widget buildRoutineList() {
139   if (_selectedDay == null) {
140     return const SizedBox.shrink();
141   }
142
143   if (_routines.isEmpty) {
144     return const Padding(
145       padding: EdgeInsets.all(16.0),
146       child: Text('No routines scheduled for the selected day'),
147     );
148   }
149
150   return Expanded(
151     child: ListView.builder(
152       itemCount: _routines.length,
153       itemBuilder: (context, index) {
154         final routine = _routines[index];
155         final routineName = _availableRoutines.firstWhere(
156           (element) => element['RoutineID'] == routine['RoutineID'],
157           orElse: () => {'RoutineName': 'Unknown Routine'},
158         )['RoutineName'];
159
160         return ListTile(
161           title: Text(routineName),
162           onTap: () {
163             // Navigate to RoutineInfoScreen and pass the routine data
164             Navigator.push(
165               context,
166               MaterialPageRoute(
167                 builder: (context) => RoutineInfoScreen(
168                   routineName: routineName,
169                   routineId: routine['RoutineID'],
170                 ),
171             ),
172           );
173         },
174       );
175     },
176   );
177 };
178 }
179

```

```

180 @override
181 Widget build(BuildContext context) {
182   return Scaffold(
183     appBar: AppBar(
184       title: const Text('Routine Scheduler'),
185     ),
186     body: Column(
187       children: [
188         TableCalendar(
189           firstDay: DateTime.utc(2020, 1, 1),
190           lastDay: DateTime.utc(2030, 12, 31),
191           focusedDay: _focusedDay,
192           selectedDayPredicate: (day) => isSameDay(_selectedDay, day),
193           onDaySelected: (selectedDay, focusedDay) {
194             setState(() {
195               _selectedDay = selectedDay;
196               _focusedDay = focusedDay;
197             });
198             fetchScheduledRoutines();
199           },
200           calendarFormat: CalendarFormat.month,
201         ),
202         buildRoutineList(),
203       ],
204     ),
205   );
206 }
207 }

```

7.11 Schedule Dialog

The `schedule_dialog.dart` file contains the code for the popup when scheduling a routine for a day. It also handles adding a planned routine to the database, so that the calendar can access it

```

1 import 'package:flutter/material.dart';
2 import 'package:http/http.dart' as http;
3 import 'dart:convert';
4 import 'package:intl/intl.dart';
5
6 class ScheduleDialog extends StatefulWidget {
7   final int routineId;
8
9   const ScheduleDialog({super.key, required this.routineId});
10
11   @override

```

```

12  _ScheduleDialogState createState() => _ScheduleDialogState();
13 }
14
15 class _ScheduleDialogState extends State<ScheduleDialog> {
16   DateTime _selectedDate = DateTime.now();
17   bool _isRepeat = false;
18   String _repeatFrequency = 'None';
19   DateTime? _endDate;
20
21   Future<void> _scheduleRoutine() async {
22     DateTime currentDate = _selectedDate;
23     while (currentDate
24       .isBefore(_endDate ?? _selectedDate.add(const Duration(days: 1)))) {
25       final response = await http.post(
26         Uri.parse('http://localhost:8080/routine/schedule'),
27         headers: {'Content-Type': 'application/json'},
28         body: json.encode({
29           'RoutineID': widget.routineId,
30           'ScheduledDate': currentDate.toIso8601String(),
31         })),
32       );
33
34       if (response.statusCode != 200) {
35         ScaffoldMessenger.of(context).showSnackBar(
36           const SnackBar(content: Text('Failed to schedule routine')),
37         );
38         return;
39       }
40
41       // calculate next date based on frequency
42       switch (_repeatFrequency) {
43         case 'Daily':
44           currentDate = currentDate.add(const Duration(days: 1));
45           break;
46         case 'Weekly':
47           currentDate = currentDate.add(const Duration(days: 7));
48           break;
49         case 'Monthly':
50           currentDate = DateTime(
51             currentDate.year, currentDate.month + 1, currentDate.day);
52           break;
53         default:
54           currentDate = _endDate != null
55             ? _endDate!.add(const Duration(days: 1))
56             : currentDate;
57           break;
58       }
59
60       // stop if there is no repetition set
61       if (!_isRepeat) break;
62     }
63

```

```

64     Navigator.pop(context);
65   }
66
67   @override
68   Widget build(BuildContext context) {
69     return AlertDialog(
70       title: const Text('Schedule Routine'),
71       content: Column(
72         mainAxisAlignment: MainAxisAlignment.min,
73         children: [
74           ListTile(
75             title: const Text('Date'),
76             subtitle: Text(DateFormat.yMd().format(_selectedDate)),
77             trailing: IconButton(
78               icon: const Icon(Icons.calendar_today),
79               onPressed: () async {
80                 final selected = await showDatePicker(
81                   context: context,
82                   initialDate: _selectedDate,
83                   firstDate: DateTime.now(),
84                   lastDate: DateTime(2100),
85                 );
86                 if (selected != null) {
87                   setState(() {
88                     _selectedDate = selected;
89                   });
90                 }
91               },
92             ),
93           ),
94           SwitchListTile(
95             title: const Text('Repeat'),
96             value: _isRepeat,
97             onChanged: (bool value) {
98               setState(() {
99                 _isRepeat = value;
100             });
101           },
102         ),
103         if (_isRepeat)
104           DropdownButton<String>(
105             value: _repeatFrequency,
106             items: ['None', 'Daily', 'Weekly', 'Monthly']
107               .map((freq) =>
108                 DropdownMenuItem(value: freq, child: Text(freq)))
109               .toList(),
110             onChanged: (value) {
111               setState(() {
112                 _repeatFrequency = value!;
113             });
114           },
115         ),

```

```

116         if (_isRepeat)
117             ListTile(
118                 title: const Text('End Date'),
119                 subtitle: _endDate != null
120                     ? Text(DateFormat.yMd().format(_endDate!))
121                     : const Text('No end date'),
122                 trailing: IconButton(
123                     icon: const Icon(Icons.calendar_today),
124                     onPressed: () async {
125                         final selected = await showDatePicker(
126                             context: context,
127                             initialDate: _endDate ?? _selectedDate,
128                             firstDate: _selectedDate,
129                             lastDate: DateTime(2100),
130                         );
131                         if (selected != null) {
132                             setState(() {
133                                 _endDate = selected;
134                             });
135                         }
136                     },
137                 ),
138             ),
139     ],
140 ),
141 actions: [
142     TextButton(
143         onPressed: () => Navigator.pop(context),
144         child: const Text('Cancel'),
145     ),
146     TextButton(
147         onPressed: _scheduleRoutine,
148         child: const Text('Schedule'),
149     ),
150 ],
151 );
152 }
153 }
154

```